
Sparse Graph Convolution

Release 0.3.6

Luke Lowery

Jan 19, 2026

CONTENTS

1	Overview	1
1.1	Installation	1
1.2	Basic Example	1
1.3	Citation & Acknowledgements	2
2	Usage Guide	3
2.1	Quickstart	3
2.2	Choosing a Graph	4
2.3	Signals	4
2.4	Kernel Functions	5
2.5	Chebyshev Approximation	7
2.6	Dynamic Graphs	8
3	Theory	9
3.1	Graph Laplacian	9
3.2	Analytical Filters	11
3.3	Kernel Fitting	11
4	Examples	13
4.1	Static Graphs	13
4.2	Dynamic Graphs	19
4.3	Spectral Graph Modal Analysis	21
4.4	3D Meshes for Visual Understanding	26
4.5	Benchmarks	27
5	Library	31
5.1	Laplacians	31
5.2	Signals	31
5.3	Kernels	32
5.4	Available Data Summary	33
5.5	Kernel JSON Structure	33
6	API Reference	35
6.1	Graph Convolution	35
6.2	Applications	39
6.3	Utility	42
7	Development	47
7.1	Validation Tests	47
7.2	Changelog	47

8 Glossary	51
Index	53

OVERVIEW

A high-performance Python library for sparse Graph Signal Processing (GSP) and Spectral Graph Wavelet Transforms (SGWT). This package leverages the CHOLMOD library for efficient sparse direct solvers, providing significant speedups over traditional dense or iterative methods for large-scale graph convolution.

Some of the key features include:

- **High-Performance Sparse Solvers:** Direct integration with the CHOLMOD library for optimized sparse Cholesky factorizations and linear system solves.
- **Generalized Graph Convolution:** Support for arbitrary spectral kernels via rational approximation (Kernel Fitting), polynomial approximation (Chebyshev), and standard analytical filters (low-pass, band-pass, high-pass).
- **Dynamic Topology Support:** Specialized routines for graphs with evolving structures, utilizing efficient rank-1 updates for real-time topology changes.
- **Resource-Aware Execution:** Context-managed memory allocation and workspace reuse to minimize overhead in high-throughput applications.
- **Integrated Graph Repository:** Built-in access to standardized graph Laplacians and signals from power systems and infrastructure networks.

For detailed usage, API reference, and theoretical background, please visit the [documentation website](#).

1.1 Installation

The `sgwt` package requires Python 3.7+ and is currently only compatible with **Windows** operating systems due to its reliance on a pre-compiled CHOLMOD library.

Install the latest stable release from [PyPI](#):

```
pip install sgwt
```

This command will also install the necessary dependencies (e.g., NumPy, SciPy).

1.2 Basic Example

Here is a quick example using a band-pass filter to an impulse signal on the synthetic Texas grid to get the wavelet function at three different scales.

```
import sgwt  
  
# Graph Laplacian
```

(continues on next page)

(continued from previous page)

```
L = sgwt.DELAY_TEXAS

# Impulse at 600th Vertex
X = sgwt.impulse(L, n=600)

with sgwt.Convolve(L) as conv:

    # Wavelet at 3 scales
    Y = conv.bandpass(X, scales=[0.1, 1, 10])
```

The `examples/` directory contains a comprehensive suite of demonstrations, also rendered in the [Examples](#) section of the documentation. Key applications include:

- **Static Filtering:** Basic low-pass, band-pass, and high-pass filtering on various graph sizes.
- **Dynamic Graphs:** Real-time topology updates, performance comparisons, and online stream processing.

1.3 Citation & Acknowledgements

If you use this library in your research, please cite it. The [GitHub repository](#) includes a `CITATION.cff` file that provides citation metadata. On GitHub, you can use the “Cite this repository” button on the sidebar to get the citation in your preferred format (including BibTeX).

For convenience, the BibTeX entry for the associated paper is:

```
@inproceedings{lowery-sgwt-2026,
  title={Using Spectral Graph Wavelets to Analyze Large Power System Oscillation Modes},
  author={Lowery, Luke and Baek, Jongoh and Birchfield, Adam},
  year={2026}
}
```

Luke Lowery developed this module during his PhD studies at Texas A&M University. You can learn more on his [research page](#) or view his publications on [Google Scholar](#).

An alternative implementation in [Julia](#) is also available and leverages native SuiteSparse support.

- The core performance of this library relies on the CHOLMOD library from [SuiteSparse](#), developed by Dr. Tim Davis at Texas A&M University.
- The graph laplacians used in the examples are derived from the [synthetic grid repository](#), made available by Dr. Adam Birchfield at Texas A&M University.

USAGE GUIDE

2.1 Quickstart

This guide demonstrates the fundamental workflow for filtering a signal on a graph using `sgwt`. We will load a built-in graph, create a test signal, and apply a multi-scale band-pass filter.

```
from sgwt import Convolve, impulse
from sgwt import DELAY_TEXAS as L

# Load the graph Laplacian for the synthetic Texas power grid.
# Create a test signal: a single impulse on vertex 600.
signal = impulse(L, n=600)

# Define the scales for the band-pass filter.
scales = [0.1, 1.0, 10.0]

# Use the Convolve context for efficient filtering.
with Convolve(L) as conv:
    # Apply the filter.
    filtered_signals = conv.bandpass(signal, scales)

# The output is a list of signals, one for each scale.
print(f"Graph has {L.shape[0]} vertices.")
print(f"Filtered signal for scale={scales[0]}: shape={filtered_signals[0].shape}")
print(f"Filtered signal for scale={scales[1]}: shape={filtered_signals[1].shape}")
print(f"Filtered signal for scale={scales[2]}: shape={filtered_signals[2].shape}")
```

2.1.1 What's Happening in This Example?

1. **Load the Graph and Signal** - `from sgwt import DELAY_TEXAS as L`: We import a pre-built graph Laplacian for the **synthetic Texas** power grid. This matrix `L` defines the graph's structure. - `signal = impulse(L, n=600)`: We create a simple test signal. It's a Dirac impulse, with a value of `1.0` on vertex 600 and `0.0` everywhere else. This is useful for seeing how a filter "spreads" energy across the graph.
2. **Define Filter Parameters** - `scales = [0.1, 1.0, 10.0]`: We specify the scales for our band-pass filter. In spectral graph filtering, the scale is inversely related to frequency. Smaller scales target high-frequency components (details, sharp changes), while larger scales target low-frequency components (smooth variations).
3. **Perform the Convolution** - `with Convolve(L) as conv::` This is the core of the library. We create a `Convolve` context. Upon entry, it performs an efficient one-time symbolic factorization of the graph Laplacian `L`. This pre-computation makes all subsequent filtering operations extremely fast. The context also manages all the low-level memory required by the CHOLMOD backend. - `filtered_signals = conv.bandpass(signal,`

scales): Inside the context, we call the `bandpass()` method. It solves the underlying linear systems to apply the filter kernel at each of the specified scales.

4. **Inspect the Output** - The result, `filtered_signals`, is a Python list where each element is a NumPy array corresponding to the filtered signal at one of the input scales. All arrays have the same shape as the input `signal`.

➡ See also

- [Signals](#) for signal formatting requirements.
- [Kernel Functions](#) for details on filter types.
- [Theory](#) for the mathematical background.

2.2 Choosing a Graph

The foundational element for all operations in this library is the **Graph Laplacian**. It is a matrix that mathematically represents the structure and connectivity of your graph.

The Laplacian matrix `L` must be provided as a `scipy.sparse.csc_matrix`. This sparse format is essential for the high-performance memory and computation management provided by the underlying CHOLMOD library. The matrix must be square and symmetric.

2.2.1 Built-in Graph Repository

For convenience and reproducibility, `sgwt` includes a repository of pre-built graph Laplacians from common power system test cases. These can be imported directly.

The naming convention is `METRIC_REGION`. For a full list of available graphs and metrics, see the [Laplacians](#) section.

➡ See also

- [Laplacians](#) for the complete list of built-in Laplacians.
- [Graph Laplacian](#) for the mathematical derivation and physical meaning of the weighting schemes.

```
import sgwt

# Load the Laplacian for the synthetic USA grid
# where edge weights are based on phase delay.
L_usa = sgwt.DELAY_USA

print(type(L_usa))
# Output: <class 'scipy.sparse.csc.csc_matrix'>

print(L_usa.shape)
# Output: (82223, 82223)
```

2.3 Signals

The primary input for all filtering operations is a signal defined on the vertices of the graph. This signal should be provided as a NumPy array.

2.3.1 Format and Memory Layout

The signal array X should have the shape $(n_vertices, n_timesteps)$, where:

- `n_vertices` is the number of vertices (nodes) in the graph, matching `L.shape[0]`.
- `n_timesteps` is the number of independent signals or time samples to be processed concurrently.

For optimal performance, it is highly recommended to create the signal array with **column-major (Fortran) ordering** by specifying `order='F'`. This memory layout aligns with the underlying C-based CHOLMOD library, avoiding costly data re-ordering during computation.

```
import numpy as np
from sgwt import DELAY_USA as L

n_vertices = L.shape[0]
n_signals = 10

# Create a random signal with the required ordering
X = np.random.randn(n_vertices, n_signals).astype(np.float64)
X = np.asfortranarray(X)

# Verify the memory order
assert X.flags['F_CONTIGUOUS']
```

➡ See also

Graph Laplacian

For the definition of vertex-domain functions.

2.3.2 Creating Test Signals with `impulse`

For testing and examples, the library provides the `impulse()` helper function to quickly generate a Dirac impulse (a value of 1 at one vertex and 0 everywhere else).

```
from sgwt import impulse
from sgwt import DELAY_Texas as L

# Create a signal with a single impulse at vertex 600
X_impulse = impulse(L, n=600)

print(X_impulse.shape)
# Output: (2000, 1)
```

2.4 Kernel Functions

This library supports two primary ways to define spectral graph filters: pre-defined analytical filters and fully custom filters using Vector Fitting.

2.4.1 Analytical Filters

For common signal processing tasks, three analytical filters are provided as methods on the convolution context.

- `lowpass(X, scales)`: Smooths the signal by attenuating high-frequency components.
- `bandpass(X, scales)`: Isolates components within a specific frequency band.
- `highpass(X, scales)`: Emphasizes sharp transitions by attenuating low-frequency components.

The `scales` parameter is a list of floating-point values that control the cutoff frequencies of the filters. Larger scales correspond to lower frequencies.

```
from sgwt import Convolve, impulse
from sgwt import DELAY_TEXAS as L

X = impulse(L, n=600)
scales = [0.1, 1.0, 10.0]

with Convolve(L) as conv:
    # Returns a list of filtered signals, one for each scale
    Y_lp = conv.lowpass(X, scales)
    Y_bp = conv.bandpass(X, scales)
    Y_hp = conv.highpass(X, scales)
```

➡ See also

Analytical Filters

For the mathematical definitions of these filters.

2.4.2 Custom Kernels via Vector Fitting

For advanced use cases, you can define arbitrary filter shapes using the `VFKernel` class. This approach uses a rational approximation (poles and residues) to model the desired frequency response, a technique known as Vector Fitting.

The library includes several pre-computed kernels. You can use them by passing the kernel object to `convolve()`.

```
from sgwt import Convolve, impulse, VFKernel
from sgwt import DELAY_USA as L
from sgwt import MODIFIED_MORLET

# Load a pre-computed Vector Fitting kernel
K = VFKernel.from_dict(MODIFIED_MORLET)
X = impulse(L, n=35000)

with Convolve(L) as conv:
    # Apply the custom kernel
    Y = conv.convolve(X, K)
```

This powerful feature allows you to implement specialized filters, such as the Modified Morlet wavelet used in the *Vector Fitting Kernels* example.

➡ See also

Kernel Fitting

For the theory behind rational approximation and Vector Fitting.

2.5 Chebyshev Approximation

A Chebyshev approximation represents a spectral filter $g(\lambda)$ as a sum of Chebyshev polynomials T_k :

$$g(\lambda) \approx \sum_{k=0}^N c_k T_k \left(\frac{2\lambda}{\lambda_{\max}} - 1 \right)$$

where $\lambda_{\max}$ is the largest eigenvalue of the graph Laplacian and c_k are the fitted coefficients. This allows for convolution via an efficient recurrence relation (Clenshaw's algorithm).

While `sgwt` is optimized for sparse direct solvers (rational filters), it provides a `ChebyKernel` implementation as an alternative for specific use cases and performance benchmarking.

The `ChebyKernel` class allows you to approximate arbitrary continuous functions on the graph spectrum. This is primarily used for:

1. **Performance Benchmarking:** Comparing the efficiency of polynomial recurrence against sparse direct solves.
2. **Non-Rational Kernels:** Approximating spectral shapes that are difficult to represent with rational functions.
3. **Avoiding Factorization:** In scenarios where the overhead of a sparse Cholesky factorization is undesirable.

2.5.1 Basic Usage

```
import sgwt
import numpy as np

# Define a custom filter function
def my_filter(x):
    return np.exp(-x) * np.sin(5 * x)

def f(x):
    return np.array([my_filter(x)]).T

# Initialize convolution context to get spectrum bound
conv = sgwt.ChebConvolve(L)
ubnd = conv.spectrum_bound

# Create the kernel
kernel = sgwt.ChebyKernel.from_function(f, order=50, spectrum_bound=ubnd)

# Apply convolution
with conv:
    result = conv.convolve(signal, kernel)
```

 See also

Kernel Fitting

For a comparison between polynomial and rational approximations.

2.6 Dynamic Graphs

In many real-world applications, such as power systems or communication networks, the underlying graph topology is not static. Edges (e.g., transmission lines or communication links) can be added or removed over time. Re-calculating the entire graph factorization for every small change is computationally expensive and not feasible for real-time applications.

The *DyConvolve* context is specifically designed for this scenario. It provides high-performance filtering on evolving graphs by leveraging efficient rank-1 updates to the factorization.

To achieve this performance, DyConvolve requires that the filter poles (or scales) are defined upfront and remain constant. The context pre-factors the graph for each pole, allowing subsequent topology updates and convolutions to be extremely fast.

2.6.1 Usage Example

Here is a simple example demonstrating how to initialize the context, update the topology, and see the effect on a filtered signal.

```
from sgwt import DyConvolve, impulse
from sgwt import DELAY_TEXAS as L

# 1. Define poles upfront (e.g., for a band-pass filter at scale=0.1)
poles = [1 / 0.1]

# 2. Create an impulse signal at a specific vertex
X = impulse(L, n=1200)

# 3. Use the DyConvolve context
with DyConvolve(L, poles) as conv:
    # Filter the signal on the original graph
    Y_before = conv.bandpass(X)

    # 4. Introduce a topology change: add a new edge
    # This connects vertex 1200 and 600 with a high weight.
    conv.addbranch(1200, 600, w=100.0)

    # 5. Filter the same signal again on the modified graph
    Y_after = conv.bandpass(X)
```

In this example, `Y_after` will show the impulse signal having propagated from vertex 1200 to vertex 600, which would not have occurred in `Y_before`.

For a more advanced simulation of a real-time data stream with topology events, see the *Online Dynamic Processing* example.

See also

Kernel Fitting

For details on the underlying Cholesky-based solver algorithms.

THEORY

This section provides the mathematical and algorithmic foundations of the Sparse SGWT library. It covers the definition of the graph Laplacian as a discrete operator, the analytical forms of spectral filters, the rational approximation of custom kernels via Vector Fitting, and the implementation details of the high-performance Cholesky-based solvers that enable scalable graph convolutions.

3.1 Graph Laplacian

This section outlines the mathematical foundations of the Sparse SGWT library.

Let an undirected graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}, \mathbf{A}, \mathbf{w}\}$ be defined by a set of vertices $|\mathcal{V}| = N$ and a set of edges $\mathcal{E}$ which are related by the arc-node incidence matrix $\mathbf{A} \in \mathbb{R}^{|\mathcal{E}| \times |\mathcal{V}|}$ and the vector of branch weights $\mathbf{w} \in \mathbb{R}^{|\mathcal{E}|}$.

A *vertex domain* function on the graph $f : \mathcal{V} \rightarrow \mathbb{R}$ can be written as a vector $\mathbf{f} \in \mathbb{R}^N$, whose i^{th} element corresponds to the evaluation of f at the i^{th} vertex.

The **Graph Laplacian** is denoted by $\mathbf{L} \in \mathbb{R}^{N \times N}$, a discrete analogue of the continuous Laplace-Beltrami operator:

$$\mathbf{L} := \mathbf{A}^\top \text{diag}(\mathbf{w}) \mathbf{A}$$

Alternatively, this can be viewed element-wise as $\mathbf{L} = \mathbf{D} - \mathbf{W}$, where $\mathbf{D}$ is the degree matrix and $\mathbf{W}$ is the weighted adjacency matrix.

3.1.1 The Physics of the Weights

While any symmetric matrix can serve as a Laplacian, `sgwt` utilizes specific weighting schemes to ensure the graph spectral domain aligns with the physical behavior of power grids.

When working with a physical distance metric, we might want to weigh the branches by the **Inverse Squared Length**. For each branch of length ℓ_{ij} , the corresponding branch weight is assigned as:

$$w_{ij} = \frac{1}{\ell_{ij}^2}$$

This weighting is not arbitrary. By defining the weights as ℓ^{-2} , the eigenvalues of the graph Laplacian (λ) correspond directly to the **squared wavenumber** (k^2) of traveling waves on the grid.

- **Low Eigenvalues:** Correspond to long-wavelength, inter-area oscillations that span the entire continent.
- **High Eigenvalues:** Correspond to short-wavelength, local disturbances.

When we apply the SGWT to this graph, each scale $a \in \mathcal{A}$ becomes physically meaningful, corresponding to a squared *pseudo-wavelength*, $r \in \mathbb{R}$, where $a = r^2$ defines this mapping. This allows for filtering based on physical spread rather than just temporal frequency.

3.1.2 Derivation of the Delay Weight

To derive the precise branch weights for the **Delay Graph Laplacian**, we must calculate the effective parameters of transformers and transmission lines, accounting for the fact that real-world grid data often aggregates these values.

Effective Branch Parameters

1. **Inductance:** We derive the branch inductance L_{ij} from the imaginary part of the series impedance Z_{ij}^{br} :

$$\omega L_{ij} = \text{Im} (Z_{ij}^{br})$$

2. **Capacitance:** Shunt admittance is often stored as a nodal aggregate. We first calculate the net nodal capacitance C_n from the shunt admittance Y_n^{sh} :

$$\omega C_n = \text{Im} (Y_n^{sh})$$

To apply this to the edges (consistent with π -model assumptions), we define the **effective branch capacitance** C_{ij} as the average of the nodal capacitances at the branch terminals:

$$C_{ij} = \frac{1}{2}(C_i + C_j)$$

Lossless Propagation Delay

Using these parameters, we estimate the lossless propagation delay θ_{ij} for every branch. This represents the “electrical distance” a wave must travel:

$$\theta_{ij} = \beta_{ij} \ell_{ij} = \omega \tau_{ij} = \text{Re} \left(\sqrt{Y_{ij} Z_{ij}} \right)$$

The expression above works because the total line parameters of the line are given, rather than the per-meter values. This forms the definition of the **Delay Graph Laplacian**:

$$\mathbf{L}_\tau = \mathbf{A}^\top \text{diag}(\theta^{-2}) \mathbf{A}$$

Which has shown to be one of the most reliable branch weights in practice.

3.1.3 Summary of Useful Weighting Schemes

Depending on the physical phenomenon being analyzed, different weighting schemes may be appropriate.

Reciprocal Squared Delay (Preferred):

$$w_{ij} = \frac{1}{\theta_{ij}^2}$$

Used for **wave propagation** analysis or dynamics. The determination for the value of τ in the previous section has proven practical for very large synthetic cases and results in a well-behaved network that outperformed other branch metrics in SGWT applications like modal analysis and FOSL.

Reciprocal Squared Distance:

$$w_{ij} = \frac{1}{d_{ij}^2}$$

Used for **geographic analysis** when electrical parameters are unavailable but GPS coordinates are known. Aligns the graph spectrum with the squared wavenumber (k^2).

Admittance Magnitude:

$$w_{ij} = \left| \frac{1}{Z_{ij}} \right| = |Y_{ij}|$$

Used where circuit analysis is directly concerned.

3.2 Analytical Filters

3.2.1 Low-Pass Spectral Graph Filter

The low-pass filter is a refinable, self-similar rational function useful for signal smoothing. The *scale* parameter, s , controls the cutoff frequency.

$$\phi_s(\Lambda) = \frac{I}{s\Lambda + I}$$

3.2.2 High-Pass Spectral Graph Filter

The high-pass filter isolates high-frequency variations on the graph.

$$\mu_s(\Lambda) = \frac{s\Lambda}{s\Lambda + I}$$

3.2.3 Band-Pass Spectral Graph Filter

A convenient closed-form wavelet generating kernel that serves as an alternative to custom-designed filters. The scale s adjusts the center frequency of the band.

$$\Psi_s(\Lambda) = \frac{4s\Lambda}{(s\Lambda + I)^2}$$

This filter qualifies as a wavelet generating kernel for the SGWT, since $\Psi(0) = 0$ and the admissibility condition is satisfied. The admissibility constant of this band-pass filter is $C_\Psi = 8/3$.

$$\Psi(0) = 0 \quad \text{and} \quad \int_0^\infty \frac{\Psi^2(x)}{x} dx < \infty$$

See also

The analytical filters are implemented as methods on the convolution contexts:

- `lowpass()`
- `bandpass()`
- `highpass()`

3.3 Kernel Fitting

3.3.1 From Polynomials to Linear Solves

In classical Graph Signal Processing, filters are often approximated using **Chebyshev polynomials**. To apply a filter $g(\mathbf{L})$, one computes a polynomial of the matrix $\mathbf{L}$. While effective for small graphs, this method has a major drawback:

it requires repeated matrix-vector multiplications. For sharp filters (like an ideal band-pass), the polynomial degree must be very high (often >50), making the calculation slow.

sgwt utilizes **Rational Approximation (Kernel Fitting)** to bypass this bottleneck.

Instead of a polynomial, we approximate the filter kernel $g(\lambda)$ as a sum of partial fractions (poles and residues):

$$g(\lambda) \approx d + \sum_{k=1}^K \frac{r_k}{\lambda + q_k}$$

The Computational Advantage This mathematical transformation changes the nature of the computation. Applying the filter no longer relies on high-degree matrix multiplication. Instead, it becomes a series of **linear system solves**:

$$(\mathbf{L} + q_k \mathbf{I}) \mathbf{y}_k = \mathbf{x}$$

Because power grid graphs are physically sparse (buses are only connected to a few neighbors), these linear systems are extremely sparse. We can solve them efficiently using **Sparse Cholesky Factorization**. This is better the polynomial approaches, due to:

1. **Speed:** For large, sparse matrices, a Cholesky solve is orders of magnitude faster than the dense operations required by high-degree polynomials.
2. **Precision:** Rational functions can approximate sharp transitions (like step functions) with far fewer terms than polynomials.

This architecture allows sgwt to perform complex filtering operations on grids with 80,000+ nodes in seconds.

For the context of large-scale power systems, this **direct solve approach** is preferred to iterative approximations. Time-varying graph signals must be processed with maximum memory efficiency to ensure scalability, especially if the process is running online.

The `cholmod_solve2` and `updown` functions serve as the primary engine, alongside various design choices that accelerate graph convolutions.

➡ See also

The `updown` functionality is exposed through the dynamic convolution context, allowing for efficient updates to the graph topology.

- [addbranch\(\)](#)

For more on the high-level concepts of graph filtering:

- [Kernel Functions](#)

The kernel fitting representation is more generally a vector fitted function, a simple pole expansion of the form:

$$\mathbf{g}(\lambda) \approx \mathbf{d} + \mathbf{e}\lambda + \sum_{k=1}^M \frac{\mathbf{r}_k}{\lambda + q_k}$$

An iterative pole reallocation procedure is used to converge to a reduced order model. The convolution of some function $\mathbf{f} * g_a$ is computed using the Cholesky decomposition and memory efficient re-factors.

➡ See also

[Kernel JSON Structure](#)

For details on the file format used to store these kernels.

EXAMPLES

The `sgwt` package includes a repository of examples:

4.1 Static Graphs

This section provides examples demonstrating static graph filtering operations and advanced usage patterns.

4.1.1 Impulse Smoothing

Demonstrates low-pass, band-pass, and high-pass filtering on the synthetic Texas grid (~2k nodes).

Listing 1: Basic Filtering on Texas Grid

```
from sgwt import Convolve, impulse
from sgwt import DELAY_TEXAS as L
from sgwt import COORD_TEXAS as C

X = impulse(L, n=600)
s = [1e-1]

with Convolve(L) as conv:
    LP = conv.lowpass(X, s)
    BP = conv.bandpass(X, s)
    HP = conv.highpass(X, s)
```



4.1.2 Filtering on East-West Grid

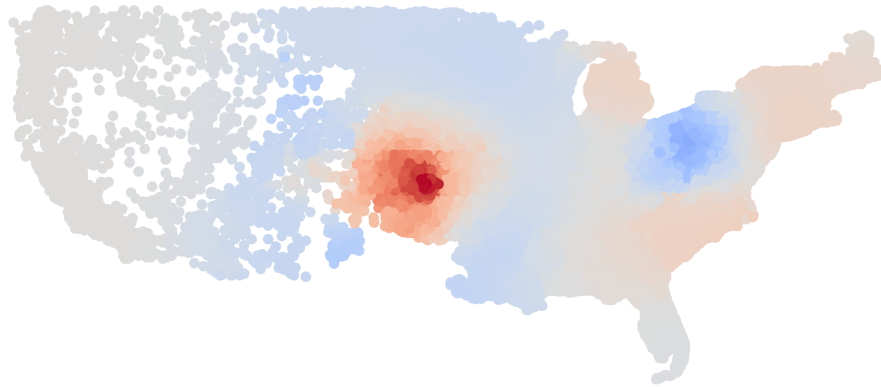
Performs filtering operations on the larger synthetic East-West US grid (~65k nodes).

Listing 2: Filtering on East-West Grid

```
from sgwt import Convolve, impulse
from sgwt import DELAY_EASTWEST as L
from sgwt import COORD_EASTWEST as C

X = impulse(L, n=65000)
X -= impulse(L, n=35000)
s = [3e0]

with Convolve(L) as conv:
    BP = conv.bandpass(X, s, order=3)[0]
```



4.1.3 Filtering on USA Grid

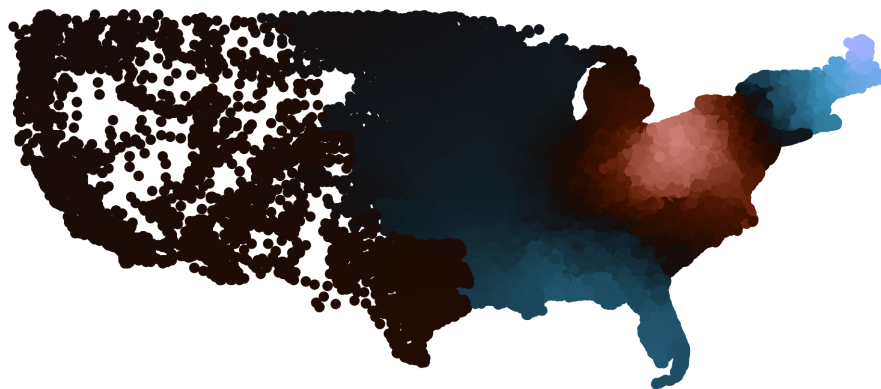
Scales up to the synthetic USA grid (~82k nodes) to demonstrate performance on larger networks.

Listing 3: Filtering on USA Grid

```
from sgwt import Convolve, impulse
from sgwt import DELAY_USA as L
from sgwt import COORD_USA as C

X = impulse(L, n=35000)
s = [10]

with Convolve(L) as conv:
    Y = conv.bandpass(X, s, order=4)[0]
```



4.1.4 Vector Fitting Kernels

Shows how to use Vector Fitting (VF) kernels to implement custom filter shapes, such as the Modified Morlet wavelet.

Listing 4: Vector Fitting Demo

```
from sgwt import DyConvolve, impulse, VFKernel
from sgwt import IMPEDANCE_EASTWEST as L
```

(continues on next page)

(continued from previous page)

```

from sgwt import COORD_EASTWEST as C
from sgwt import MODIFIED_MORLET as Kjson

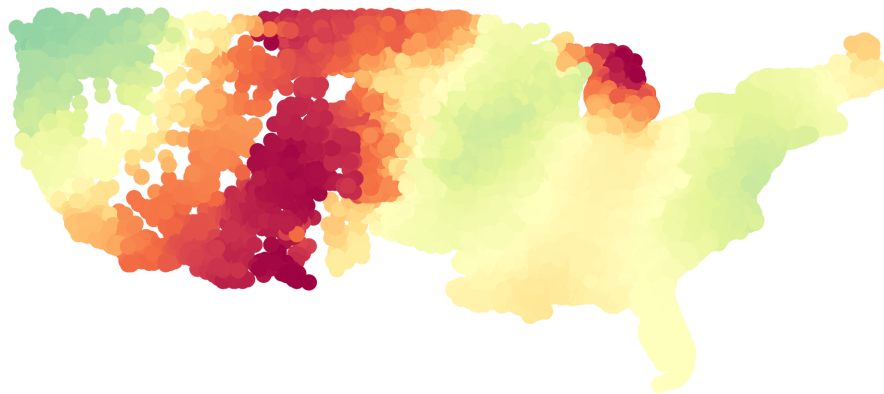
# Signal Input
X = impulse(L, n=-1000)

# NOTE: This is a temporary scaling workaround. A proper scaling method on the kernel_
↪ object should be used.
K = VFKernel.from_dict(Kjson)
K.Q /= 2000
K.R /= 2000

with DyConvolve(L, K) as g:

    Y = g.convolve(X)

```



4.1.5 Signal Inpainting

Reconstructs a smooth signal across the USA grid using only a small fraction (e.g., 0.1%) of known data points via iterative low-pass filtering.

Listing 5: Signal Inpainting

```

from sgwt import DyConvolve
from sgwt import DELAY_USA as L
from sgwt import COORD_USA as C
import numpy as np

SAMPLE_FRACTION = 0.005
N_ITERATIONS = 100
SMOOTHING_SCALE = 50.0
STEP_SIZE = 1
n_nodes = L.shape[0]

X_true = C[:, 0:1].copy(order='F')

```

(continues on next page)

(continued from previous page)

```
n_samples = int(n_nodes * SAMPLE_FRACTION)
sample_indices = np.random.choice(n_nodes, n_samples, replace=False)
J_mask = np.isin(np.arange(n_nodes), sample_indices)
X_sampled = np.zeros_like(X_true)
X_sampled[J_mask] = X_true[J_mask]

Xh = np.zeros_like(X_true, order='F')

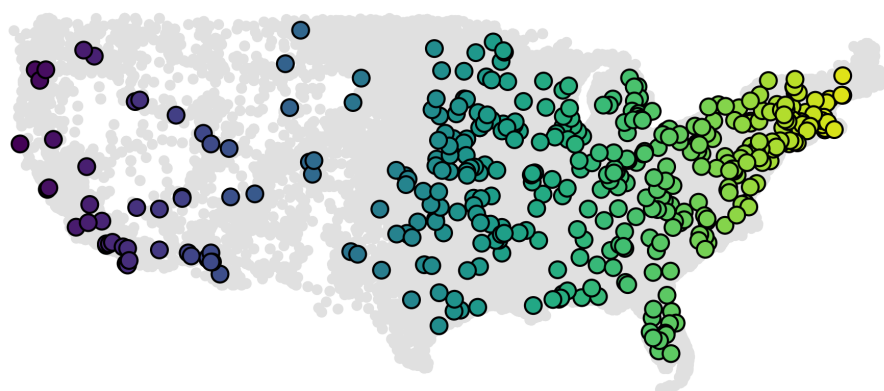
with DyConvolve(L, poles=[1/SMOOTHING_SCALE]) as conv:
    for i in range(N_ITERATIONS):
        error = np.zeros_like(Xh)
        error[J_mask] = X_sampled[J_mask] - Xh[J_mask]
        smoothed_error = conv.lowpass(error)[0]
        Xh += STEP_SIZE * smoothed_error * SMOOTHING_SCALE
```

Graph Signal Inpainting from 0.5% of Data

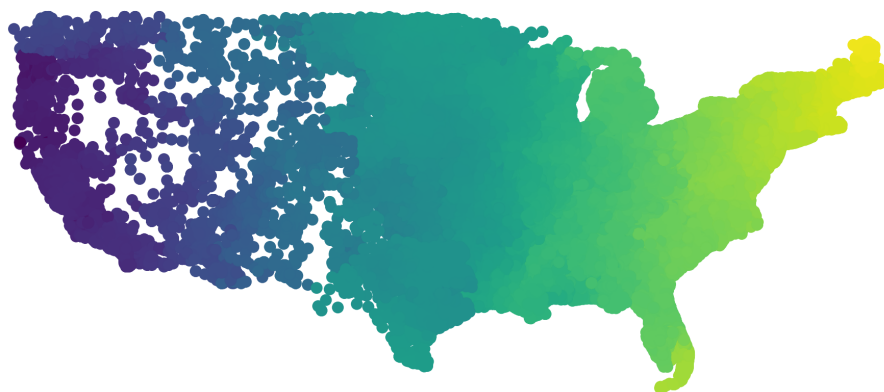
Ground Truth Signal



Input: Sparse Samples



Output: Reconstructed Signal



4.2 Dynamic Graphs

This section provides examples demonstrating the use of dynamic graph convolutions with DyConvolve.

4.2.1 Dynamic Topology

Illustrates updating the graph topology (adding branches) on-the-fly without recomputing the entire decomposition.

Listing 6: Dynamic Topology Update

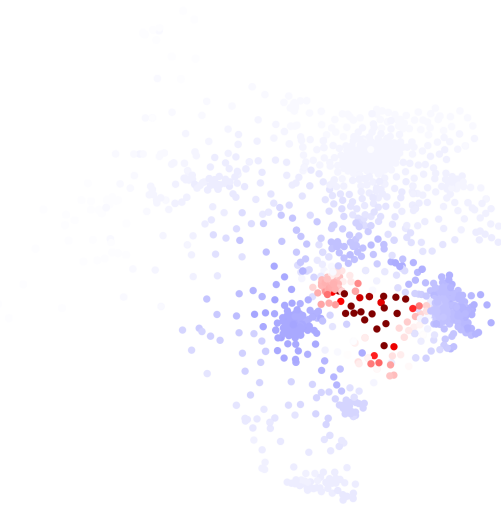
```
from sgwt import DyConvolve, impulse
from sgwt import DELAY_TEXAS as L
from sgwt import COORD_TEXAS as C

X = impulse(L, n=1200)
scales = [0.1, 1, 10]
poles = [1/s for s in scales]

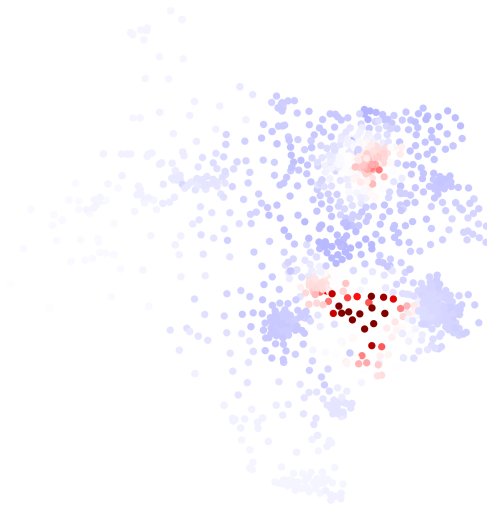
with DyConvolve(L, poles) as conv:
    Y_before = conv.bandpass(X)
    conv.addbranch(1200, 600, 1/(1e-3)**2)
    Y_after = conv.bandpass(X)
```

Dynamic Topology Update: Band-pass Filtered Signal

Before Branch Added (Bus 1200)



After Branch Added (Bus 1200 <-> 600)



4.2.2 Performance

Compares the execution time and performance between static (Convolve) and dynamic (DyConvolve) convolution methods.

Listing 7: Static vs Dynamic Performance

```
from sgwt import Convolve, DyConvolve, impulse
from sgwt import DELAY_USA as L
import numpy as np
```

(continues on next page)

(continued from previous page)

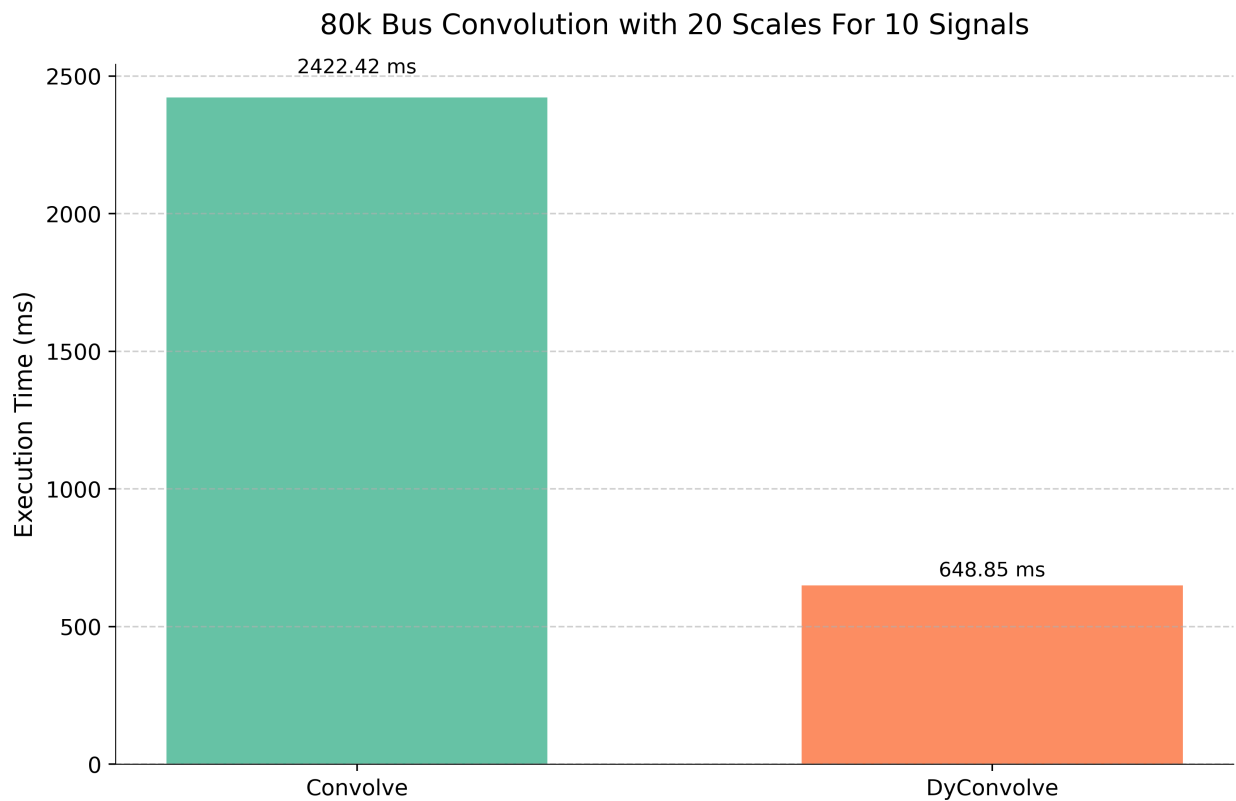
```
import time

# Impulse
X = impulse(L, n=1200)

# Pre-Determined Poles
scales = np.geomspace(1e-5, 1e2, 20)
poles = 1/scales

with Convolve(L) as conv:
    start = time.time()
    for i in range(10):
        Y = conv.bandpass(X, scales)
    T1 = time.time() - start

with DyConvolve(L, poles) as conv:
    start = time.time()
    for i in range(10):
        Y = conv.bandpass(X)
    T2 = time.time() - start
```



4.2.3 Online Dynamic Processing

Simulates an online processor handling a continuous stream of data and topology change events.

Listing 8: Dynamic Stream Processing

```
from sgwt import DyConvolve
from sgwt import DELAY_USA as L

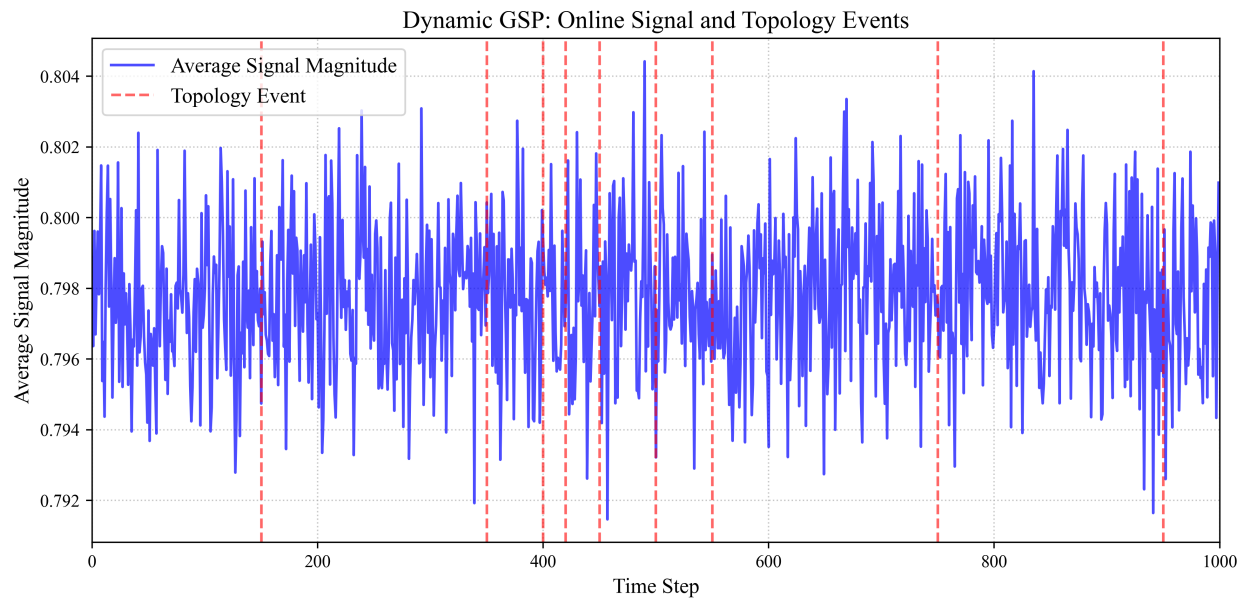
scales = np.geomspace(0.1, 10.0, 10)
poles = 1.0 / scales

avg_signal_magnitudes = []

with DyConvolve(L, poles) as conv:
    for t, f_t, event in get_incoming_data():
        if event:
            u, v, w = event
            conv.addbranch(*event)
            print(f"[{t:04d}] EVENT | Topology Update: Edge ({u} <-> {v}) added")

            # Compute wavelet coefficients
            W = conv.bandpass(f_t)
            avg_signal_magnitudes.append(np.mean(np.abs(f_t))) # Record average signal_
            ↪ magnitude

        if not event: print(f"[{t:04d}] STATUS | Stream processing active")
```



4.3 Spectral Graph Modal Analysis

This section provides examples demonstrating the use of Spectral Graph Modal Analysis (SGMA) for joint spatial-temporal wavelet transforms.

4.3.1 Local Mode Identification

Demonstrates computing the SGMA spectrum at a single bus to identify dominant oscillatory modes in the wavelength-frequency domain.

This example shows how to:

- Initialize the SGMA engine with spatial scales and temporal frequencies.
- Compute the spectrum for a specific bus and time using `sgma.spectrum()`.
- Extract and visualize dominant peaks using `sgma.find_peaks()`.

Listing 9: Single Bus SGMA Analysis

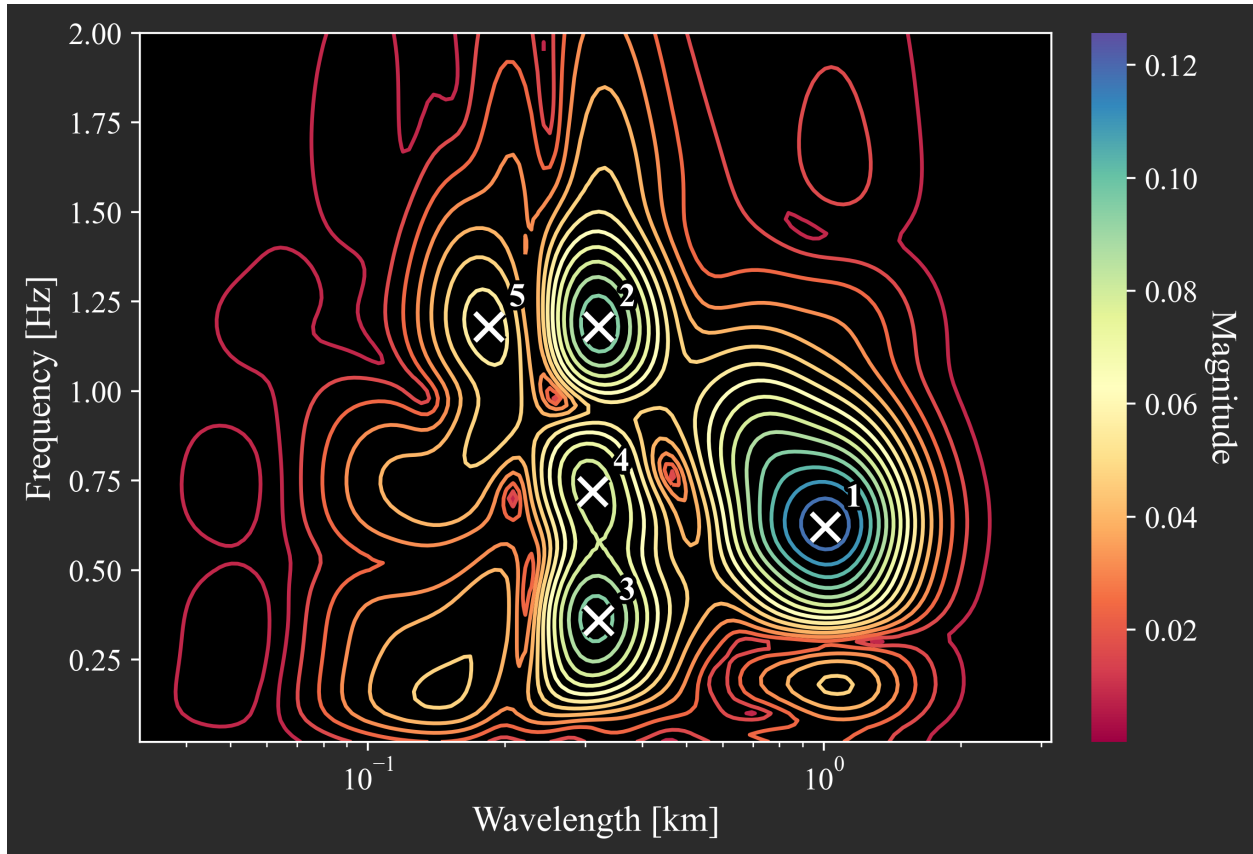
```
from sgwt import SGMA
from sgwt import DELAY_WECC as L

# Signals: Real or Complex Matrix (Rows: Buses, Cols: Time)
V, t = get_signal(FILEPATH, t_range=(0, 60))

# SGMA Parameters
BUS_TARGET = 140
TIME_TARGET = 2.0
ORDER = 10
TOP_N = 5

spatial_scales = np.geomspace(1e-3, 1e1, 150)
temporal_freqs = np.linspace(0.02, 2.0, 100)
sgma = SGMA(L, spatial_scales, temporal_freqs, order=ORDER)

M = sgma.spectrum(V, t, BUS_TARGET, TIME_TARGET)
peaks = sgma.find_peaks(M, top_n=TOP_N)
```



The resulting contour plot shows the spectrum in the wavelength-frequency domain. The overlaid markers indicate the top_n most dominant oscillatory modes (peaks) identified at the target bus and time instant.

4.3.2 Modal Analysis on a Bus Subset

Demonstrates using `analyze_many()` to efficiently identify oscillatory modes across a subset of buses.

This example shows how to:

- Analyze a random subset of buses to get a representative view of network-wide modes.
- Use the `analyze_many()` method for efficient batch processing.
- Visualize the combined peaks from all analyzed buses on a heatmap.

Listing 10: Analysis on a Subset of Buses

```
from sgwt import SGMA
from sgwt import DELAY_WECC as L

# Signals: Real or Complex Matrix (Rows: Buses, Cols: Time)
V, t = get_signal(FILEPATH, t_range=(0, 60))

# SGMA Parameters
TIME_TARGET = 2.0
N_RANDOM_BUSES = 50
ORDER = 10
TOP_N = 5
```

(continues on next page)

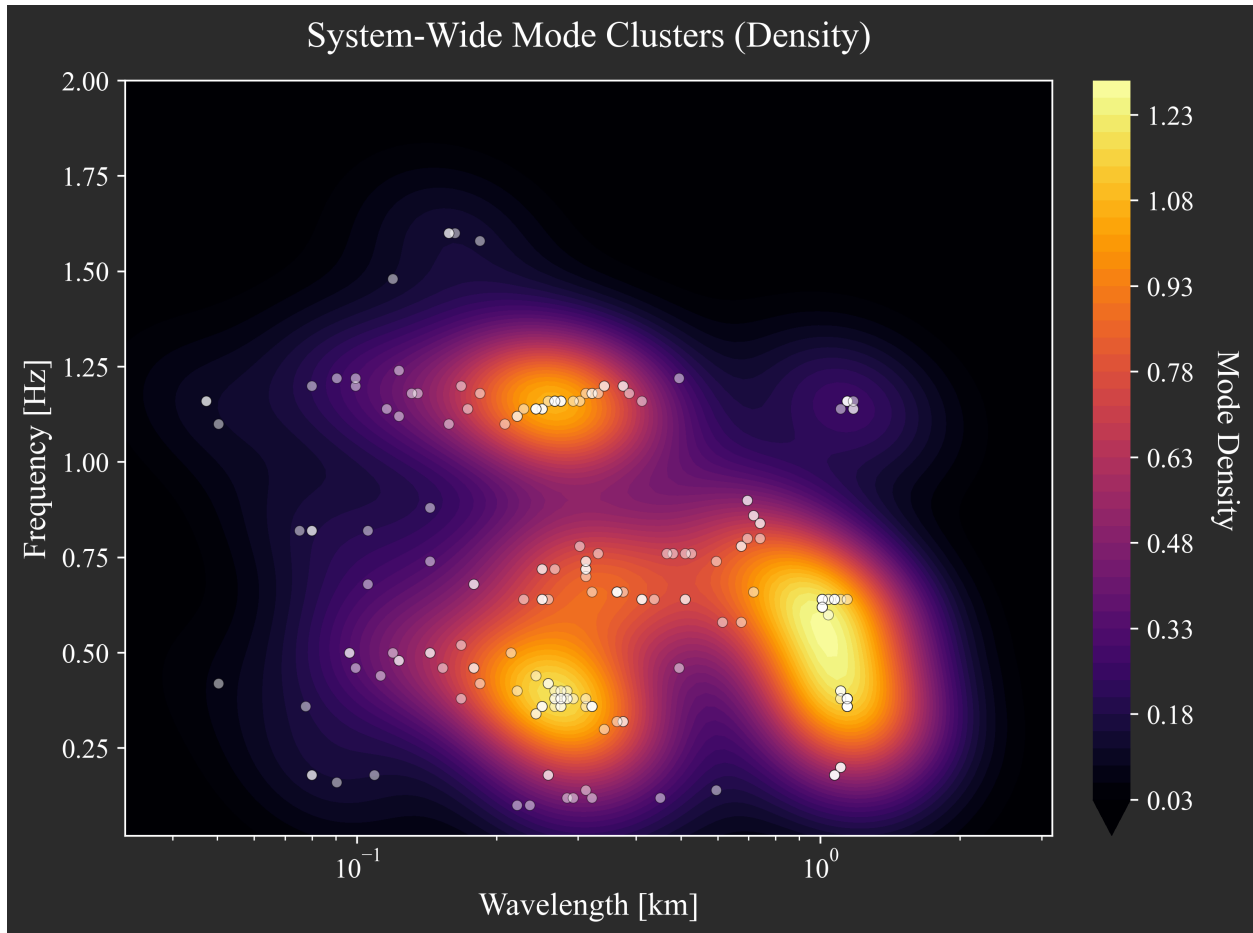
(continued from previous page)

```

spatial_scales = np.geomspace(1e-3, 1e1, 150)
temporal_freqs = np.linspace(0.02, 2.0, 100)
sgma = SGMA(L, spatial_scales, temporal_freqs, order=ORDER)

subset_buses = np.random.choice(L.shape[0], N_RANDOM_BUSES, replace=False)
result = sgma.analyze_many(V, t, time=TIME_TARGET, buses=subset_buses, top_n=TOP_N)

```



The `analyze_many()` method is highly efficient because it computes the temporal transform $V @ B$ only once and reuses it for each bus. The resulting heatmap shows all detected peaks, providing a picture of the dominant modes present in the selected subset of the network.

4.3.3 Full Network Analysis

Demonstrates using `analyze_many()` to perform a comprehensive modal analysis across all buses in a network.

This example shows how to:

- Use `analyze_many()` to compute modes for every bus in the network.
- Generate a peak density heatmap to visualize system-wide modal patterns.
- Interpret the resulting wavelength-frequency plot.

Listing 11: Full Network SGMA Analysis

```

from sgwt import SGMA
from sgwt import DELAY_WECC as L

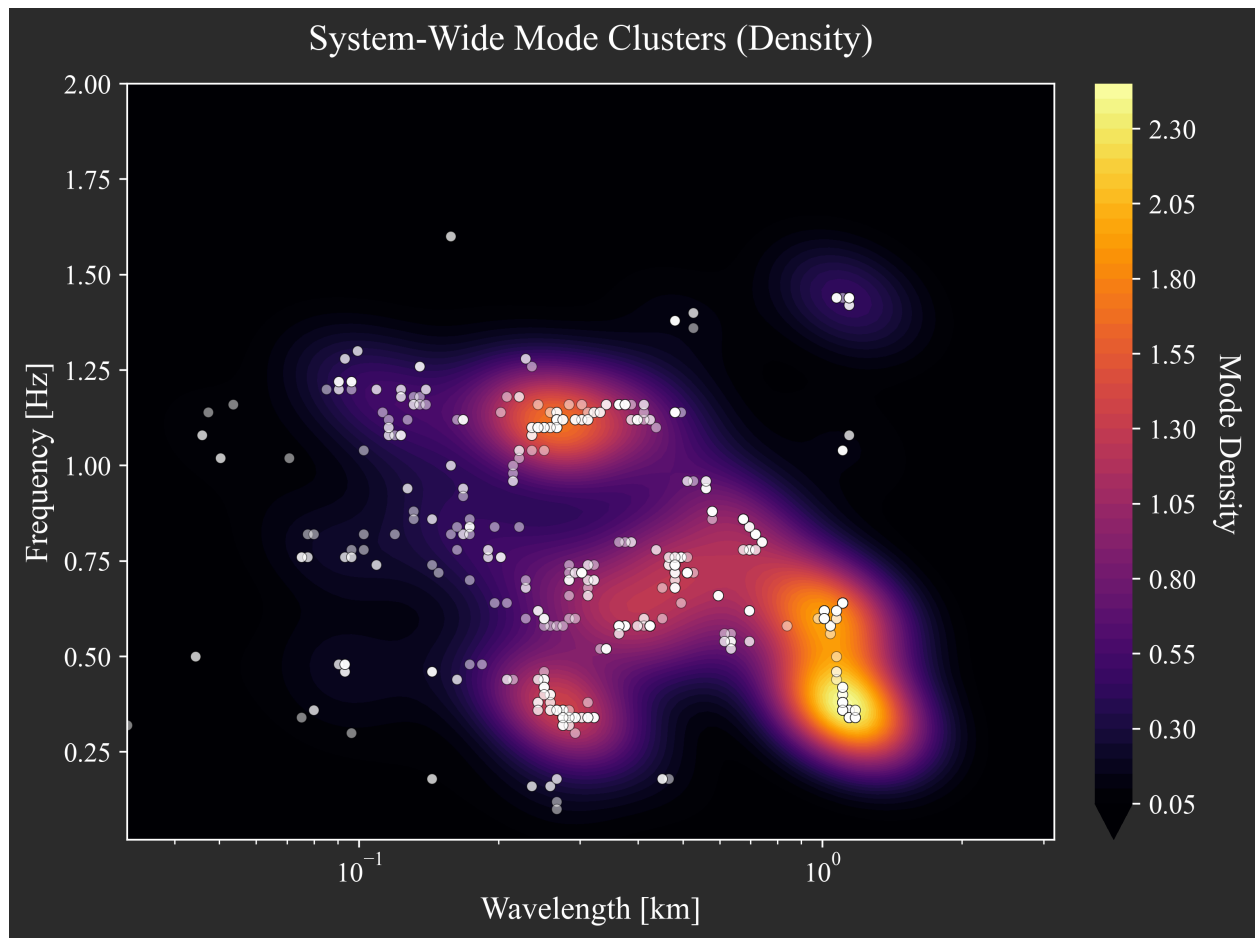
# Signals: Real or Complex Matrix (Rows: Buses, Cols: Time)
V, t = get_signal(FILEPATH, t_range=(0, 60))
V = np.abs(V)

# SGMA Parameters
TIME_TARGET = 2.0
ORDER = 10
TOP_N = 4

spatial_scales = np.geomspace(1e-3, 1e1, 150)
temporal_freqs = np.linspace(0.02, 2.0, 100)
sgma = SGMA(L, scales=spatial_scales, freqs=temporal_freqs, order=ORDER)

result = sgma.analyze_many(V, t, time=TIME_TARGET, top_n=TOP_N)

```



By calling `analyze_many()` without the `buses` argument, the analysis runs on all buses in the graph. The resulting heatmap visualizes the density of all detected peaks. Hotspots on the map indicate dominant modes that are present across many buses in the system.

- **Wavelength (y-axis):** Larger wavelengths correspond to wide-area inter-area modes, while smaller wavelengths indicate local modes.
- **Frequency (x-axis):** Identifies the temporal frequency of the oscillation in Hz.

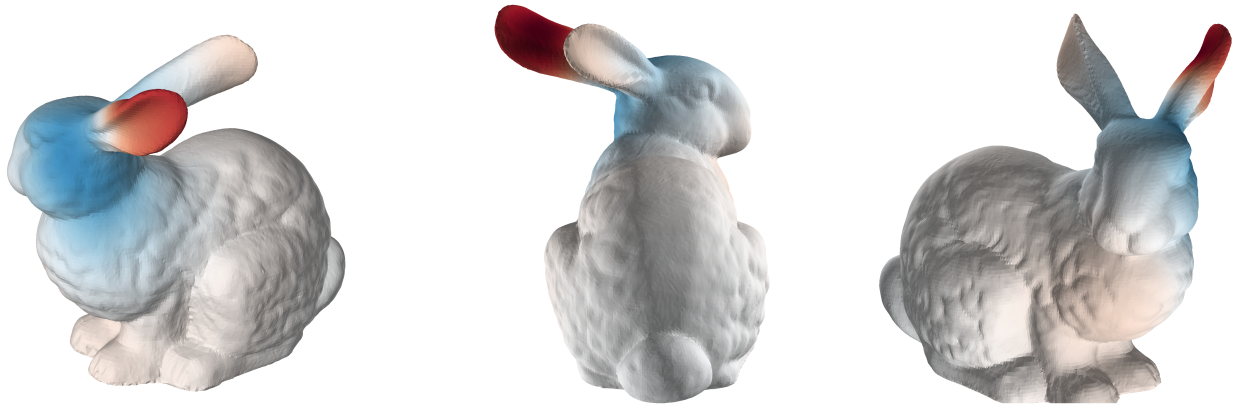
4.4 3D Meshes for Visual Understanding

Spectral graph wavelets extend naturally to 3D mesh surfaces, where the graph Laplacian captures the intrinsic geometry of the manifold. This example demonstrates wavelet localization on three mesh models of varying complexity: the Stanford Bunny, a horse, and a cortical brain surface.

For each mesh, we place an impulse signal at a single vertex and apply a band-pass filter. The `scale` parameter controls the center frequency of the filter (larger scales target lower frequencies), while the `order` parameter determines filter sharpness by repeated application of the operator. The resulting wavelet coefficients reveal how the filter's frequency response translates to spatial localization on the surface. Vertices are colored by coefficient magnitude, with the diverging colormap distinguishing positive and negative values.

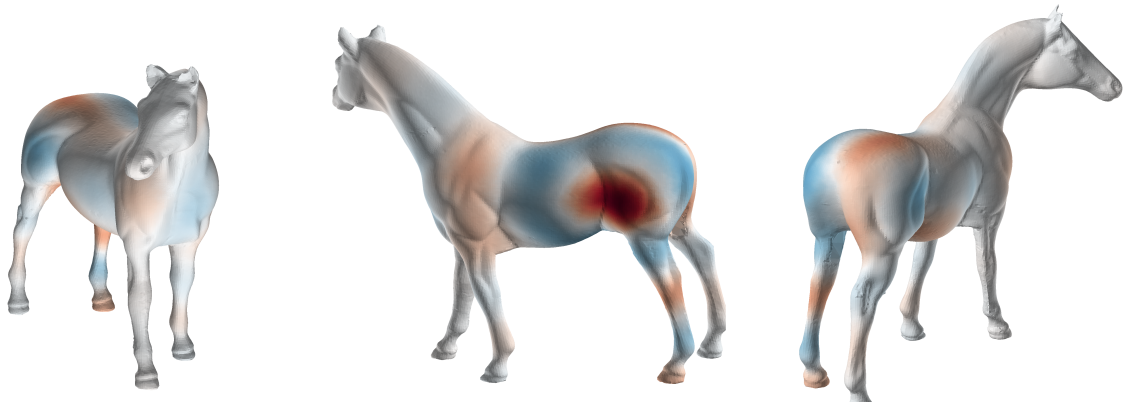
4.4.1 Stanford Bunny

The Stanford Bunny (35,947 vertices) serves as a standard benchmark in computer graphics. With a low filter order (4) at scale 200, the wavelet produces a broad, smooth response centered on vertex 15,000. The relatively gentle spectral rolloff results in gradual spatial decay across the mesh surface.



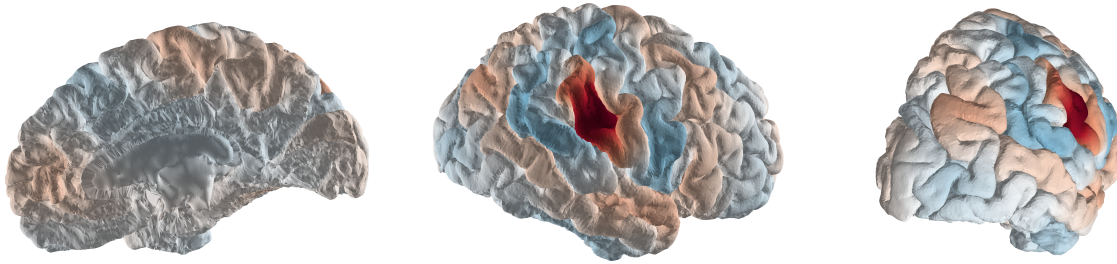
4.4.2 Horse Model

The horse mesh (48,485 vertices) presents an elongated geometry with thin limbs and a complex torso. Using a higher filter order (50) at scale 60 yields a more sharply localized wavelet. The filter response, centered at vertex 40,000, demonstrates how wavelets adapt to the local surface geometry while respecting the mesh connectivity.



4.4.3 Cortical Surface

The left hemisphere cortical surface (~150k vertices) represents a neuroimaging application where spectral methods have direct relevance. The highly folded geometry of the brain creates a graph Laplacian that encodes both spatial proximity and cortical folding patterns. The wavelet at scale 200 with order 30 shows localization that follows the cortical ribbon rather than Euclidean distance, which could be useful for multi-scale analysis of brain activity or morphometry.



4.5 Benchmarks

This section focuses on the Chebyshev polynomial approximation capabilities of `sgwt`, including accuracy analysis and performance comparisons against direct solvers.

4.5.1 Chebyshev Approximation Reference

This example demonstrates how to use `ChebConvolve` to approximate a standard band-pass filter. It serves as a reference to verify that the polynomial approximation converges to the same spatial result as the analytical solver.

Listing 12: Chebyshev Approximation vs. Analytical Solver

```
import sgwt
from sgwt import ChebyConvolve, DyConvolve, impulse
from sgwt import IMPEDANCE_Texas as L

SCALES = [0.1, 1.0, 10.0]
ORDER = 300
XMIN = 1e-3

def f(x):
    return np.stack([
        sgwt.functions.bandpass(x, scale=s, order=2) for s in SCALES],
        axis=1)

kernel = sgwt.ChebKernel.from_function_on_graph(L, f, ORDER, min_lambda=XMIN)

X = impulse(L, n=600)

with ChebyConvolve(L) as conv_cheb:
    Y_cheb = conv_cheb.convolve(X, kernel)
```

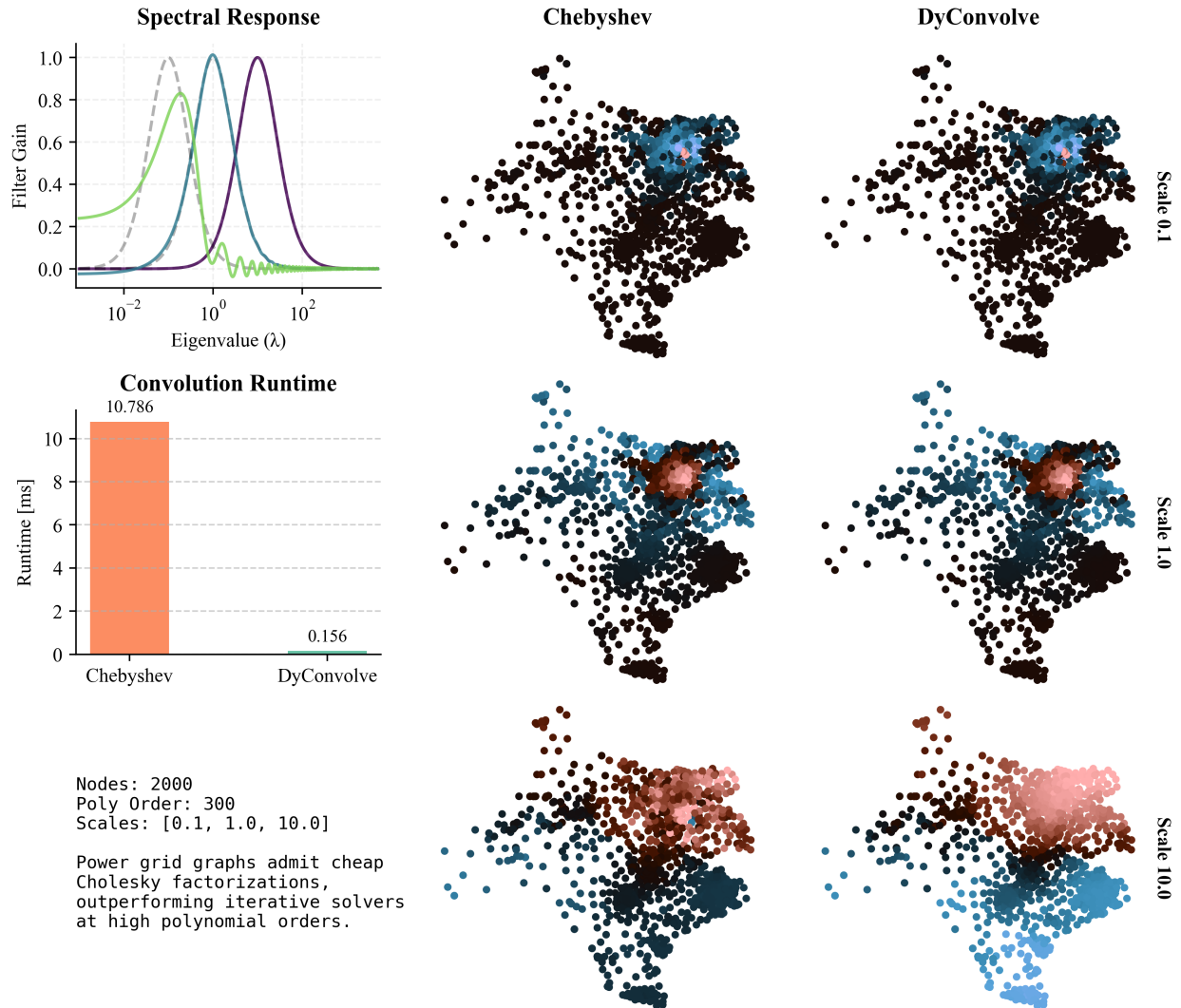
(continues on next page)

(continued from previous page)

```
with DyConvolve(L, [1.0/s for s in SCALES]) as conv:
    Y_dy = conv.bandpass(X, order=2)
```

Chebyshev vs. DyConvolve

Comparing Iterative (Chebyshev) vs. Direct (Cholesky) Solvers



4.5.2 Spectral Approximation Quality

This demo explores the relationship between polynomial order and approximation accuracy. It highlights the high order often required to match the precision of the direct rational solvers used in the core library.

Listing 13: Chebyshev Order Comparison

```
import sgwt
import numpy as np
from sgwt import IMPEDANCE_Texas as L
```

(continues on next page)

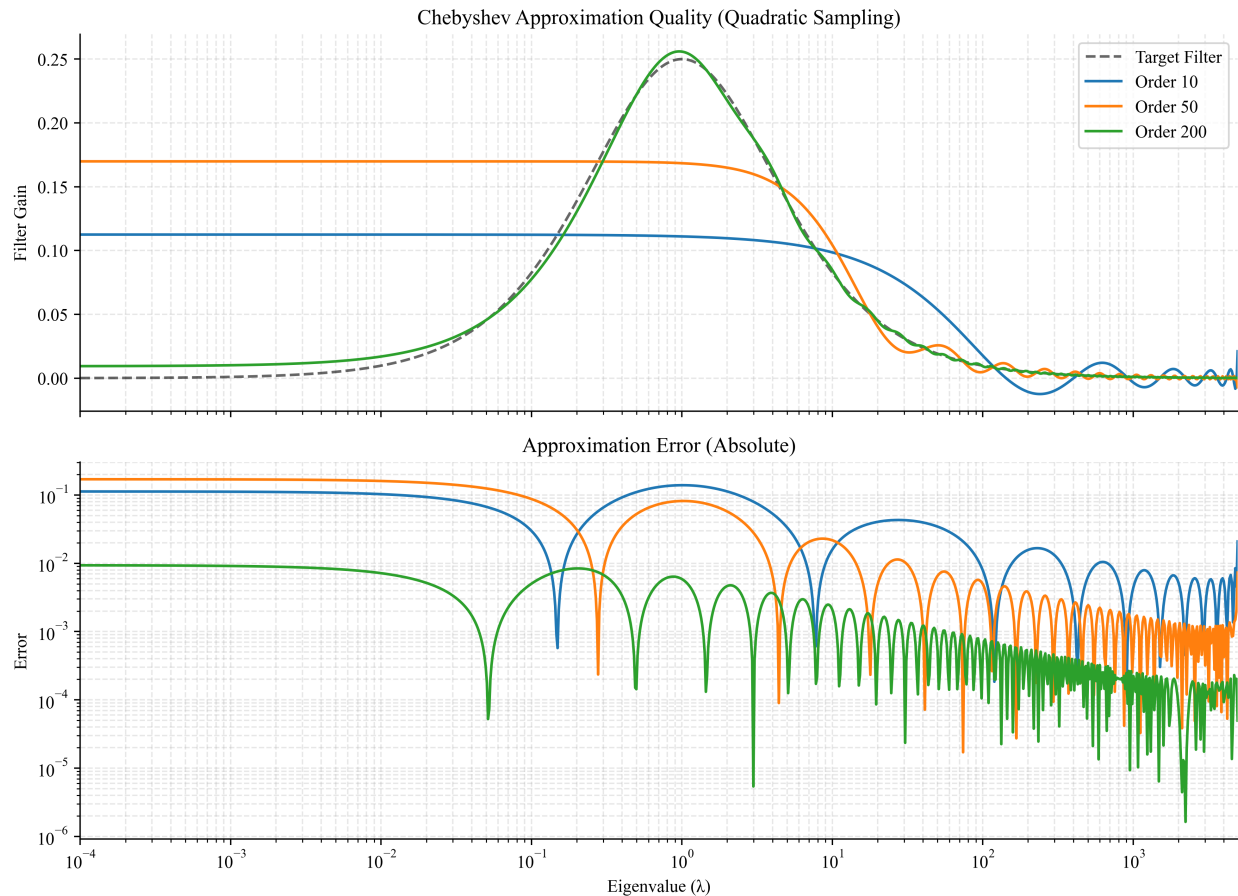
(continued from previous page)

```
def f(x): return np.array([sgwt.functions.bandpass(x, scale=1.0, order=1)]).T

ubnd = sgwt.estimate_spectral_bound(L)
x_eval = np.geomspace(1e-4, ubnd, 1000)
y_true = f(x_eval)

orders = [10, 50, 200]
```

Chebyshev Approximation Quality



4.5.3 Benchmark: Chebyshev vs. Direct Solvers

This benchmark demonstrates the performance trade-offs between polynomial approximation and sparse direct solvers. In most cases, the optimized DyConvolve solver outperforms high-order Chebyshev approximations on large sparse graphs.

Listing 14: Chebyshev vs. Analytical Timing

```
SCALES = [0.01, 0.1, 1, 10]
ORDER = 450
N_ITER = 200
```

(continues on next page)

(continued from previous page)

```

X = impulse(L, n=1200)

def f(x): return np.stack([sgwt.functions.bandpass(x, scale=s, order=1) for s in SCALES],
    ↪ axis=1)

lbnd = 1e-3
kernel = sgwt.ChebyKernel.from_function_on_graph(L, f, ORDER, min_lambda=lbnd)

print(f"Benchmarking on {L.shape[0]} nodes...")

# Time Chebyshev Convolution
with ChebyConvolve(L) as conv_cheb:
    _ = conv_cheb.convolve(X, kernel)
    start = time.time()
    for _ in range(N_ITER):
        _ = conv_cheb.convolve(X, kernel)
    t_cheb = (time.time() - start) / N_ITER

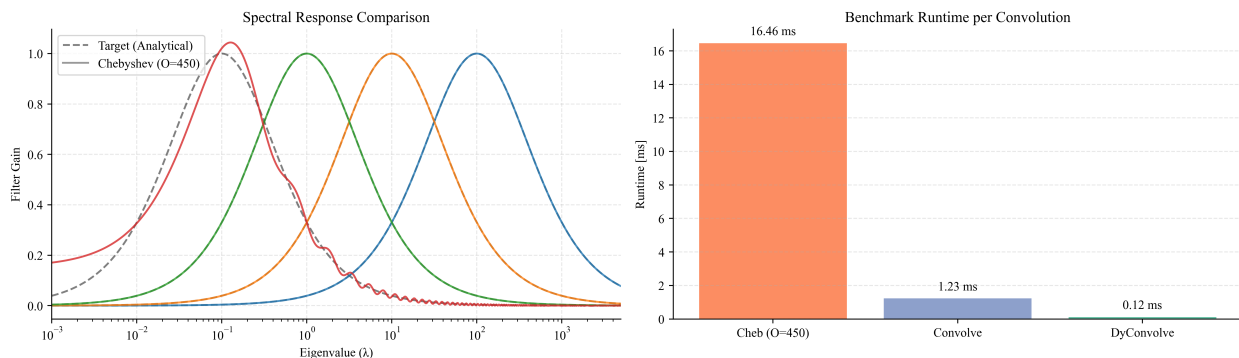
# Time DyConvolve, which pre-factors to time only the solve phase.
with DyConvolve(L, poles=[1.0/s for s in SCALES]) as conv_dy:
    _ = conv_dy.bandpass(X)
    start = time.time()
    for _ in range(N_ITER):
        _ = conv_dy.bandpass(X)
    t_analytical = (time.time() - start) / N_ITER

# Time Convolve, which includes factorization in each call.
with Convolve(L) as conv_std:
    _ = conv_std.bandpass(X, SCALES)
    start = time.time()
    for _ in range(N_ITER):
        _ = conv_std.bandpass(X, SCALES)
    t_std = (time.time() - start) / N_ITER

print(f"Chebyshev (Order {ORDER}): {t_cheb*1000:.2f} ms")
print(f"Convolve: {t_std*1000:.2f} ms")
print(f"DyConvolve: {t_analytical*1000:.2f} ms")

```

Chebyshev vs. Analytical Timing Comparison

Polynomial Order $k=450$ | Scales: 4

LIBRARY

The `sgwt` package includes a repository of built-in resources for testing, demonstration, and reproducibility. This includes standardized graph Laplacians, test signals, and pre-computed spectral kernels.

This section details the available data and the file formats used.

5.1 Laplacians

The library includes Laplacians for various synthetic power grid networks. These are provided as `scipy.sparse.csc_matrix` objects. The naming convention is `METRIC_REGION`.

See also

Graph Laplacian

For the mathematical derivation and physical interpretation of the **DELAY**, **LENGTH**, and **IMPEDANCE** weighting schemes.

Usage

```
import sgwt

# Load the Laplacian for the synthetic Texas grid
# where edge weights are based on phase delay.
L_texas = sgwt.DELAY_Texas

print(type(L_texas))
# <class 'scipy.sparse.csc.csc_matrix'>
print(L_texas.shape)
# (2000, 2000)
```

5.2 Signals

Vertex-domain signals are provided for some graphs, most commonly geographic coordinates.

- **COORDS**: An `(n_vertices, 2)` NumPy array containing the longitude and latitude of each node.

Usage

```
import sgwt
```

(continues on next page)

(continued from previous page)

```
# Load the geographic coordinates for the Texas grid
coords_texas = sgwt.COORD_TEXAS

print(type(coords_texas))
# <class 'numpy.ndarray'>
print(coords_texas.shape)
# (2000, 2)
```

5.3 Kernels

Pre-computed rational approximations for common spectral graph wavelets are available as dictionaries. These can be loaded into *VFKernel* objects for use with *convolve()*.

- **MEXICAN_HAT**: Mexican Hat wavelet.
- **MODIFIED_MORLET**: Modified Morlet wavelet.
- **SHANNON**: Shannon (ideal band-pass) wavelet.
- **GAUSSIAN_WAV**: Gaussian wavelet.

Usage

```
from sgwt import VFKernel, MODIFIED_MORLET

# The built-in kernel is a dictionary
print(type(MODIFIED_MORLET))
# <class 'dict'>

# Load it into a VFKernel object for use in convolution
kernel = VFKernel.from_dict(MODIFIED_MORLET)

print(kernel.R.shape)
# (14, 1)
```


➔ See also

See [Kernel JSON Structure](#) for details on the JSON file format for custom kernels.

5.4 Available Data Summary

The following table summarizes all available built-in datasets, including both power grid networks and 3D mesh models. The original mesh datasets can be found at the [Laplacian Library](#).

Dataset	Vertices	Edges	Vertex Signals	Edge Weights
BUNNY	35947	104288	x, y, z	
HORSE	48485	145449	x, y, z	
LBRAIN	149351	448041	x, y, z	
HAWAII	37	98	long, lat	length, impedance, delay
TEXAS	2000	2998	long, lat	length, impedance, delay
EASTWEST	78000	100923	long, lat	length, impedance, delay
USA	82000	104123	long, lat	length, impedance, delay
WECC	240	451		length, impedance, delay

5.5 Kernel JSON Structure

The library supports loading spectral kernels defined by rational approximations (Vector Fitting) stored in JSON format. These files are parsed into [VFKernel](#) objects for use in convolution.

The JSON file represents a rational expansion of the filter $g(\lambda)$ in the spectral domain, where λ represents a graph Laplacian eigenvalue. The expansion takes the form:

$$g(\lambda) \approx \mathbf{d} + \mathbf{e}\lambda + \sum_{k=1}^M \frac{\mathbf{r}_k}{\lambda + q_k}$$

This structure is derived from a Vector Fitting procedure and allows for efficient computation of the graph convolution.

5.5.1 Fields

- **description** (string, optional): A human-readable description of the kernel.
- **nfuncs** (int): The number of separate functions or dimensions in the kernel.
- **npoles** (int): The number of poles (M) in the rational approximation.
- **d** (float or list): The constant term $\mathbf{d}$ (direct feedthrough).
- **e** (float or list, optional): The linear term $\mathbf{e}$. This is often zero.
- **poles** (list): A list of pole objects, where each object contains:
 - **q** (float): A pole location q_k .
 - **r** (list): A list of residues $\mathbf{r}_k$ corresponding to the pole. The length of this list must match **nfuncs**.

Example

Below is a truncated example from the built-in `MODIFIED_MORLET.json` file.

```
{
  "description": "Modified Morlet Wavelet With Central Frequency of 2pi",
  "nfuncs": 1,
  "npoles": 14,
  "d": 0.0027966205394028575,
  "e": 0,
  "poles": [
    {
      "q": -19219.857112413505,
      "r": [-307.5212280632877]
    },
    {
      "q": -2935.9392937251964,
      "r": [132.53827518359017]
    },
    {
      "q": -56.43582644772509,
      "r": [800722.1859990739]
    }
  ]
}
```

See also

Kernel Functions

For an example of how to load and use a custom kernel in a convolution.

API REFERENCE

This page provides a detailed reference for the classes and functions within the `sgwt` package.

6.1 Graph Convolution

The core of the library consists of three specialized convolution engines designed for different use cases: static graphs, dynamic graphs with evolving topologies, and polynomial approximations.

6.1.1 Static Convolution

class `Convolve(L)`

Bases: `object`

Static graph convolution context using CHOLMOD.

Designed for high-performance GSP operations on graphs with constant topology. Manages CHOLMOD symbolic and numeric factorizations internally.

Parameters

`L` (`csc_matrix`) – Sparse Graph Laplacian of shape $(n_vertices, n_vertices)$.

➔ See also

`DyConvolve`

For graphs with evolving topologies.

Examples

```
>>> from sgwt import Convolve, LAPLACIAN_TEXAS_DELAY
>>> import numpy as np
>>> L = LAPLACIAN_TEXAS_DELAY
>>> signal = np.random.randn(L.shape[0], 100)
>>> with Convolve(L) as conv:
...     lp = conv.lowpass(signal, scales=[0.1, 1.0, 10.0])
...     bp = conv.bandpass(signal, scales=[1.0])
```

bandpass(`B`, `scales=[1]`, `order=1`)

Computes band-pass filtered wavelet coefficients at specified scales.

Uses the analytical form: $((4/s) * L / (L + I/s)^2)^{order}$.

Parameters

- **B** (`ndarray`) – Input signal array (`n_vertices`, `n_timesteps`).
- **scales** (`list[float]`, *default*: `[1]`) – List of scales to compute coefficients for.
- **order** (`int`, *default*: `1`) – The order of the filter (number of times the operator is applied).

Returns

Filtered signals for each scale.

Return type

`list[np.ndarray]`

convolve(*B*, *K*)

Performs graph convolution using a specified kernel.

Parameters

- **B** (`ndarray`) – Input signal array (`n_vertices`, `n_timesteps`) with column-major ordering (F).
- **K** (`VFKernel` | `dict`) – Kernel function (Vector Fitting model) to apply.

Returns

Convolved signal (`n_vertices`, `n_timesteps`, `nDim`).

Return type

`ndarray`

highpass(*B*, *scales*=`[1]`)

Computes high-pass filtered coefficients at specified scales.

Uses the analytical form: $sL / (sL + I)$.

Parameters

- **B** (`ndarray`) – Input signal array (`n_vertices`, `n_timesteps`).
- **scales** (`list[float]`, *default*: `[1]`) – List of scales to compute coefficients for.

Returns

Filtered signals for each scale.

Return type

`list[np.ndarray]`

lowpass(*B*, *scales*=`[1]`, *Bset*=`None`, *refactor*=`True`, *order*=`1`)

Computes low-pass filtered scaling coefficients at specified scales.

Uses the analytical form: $I / (sL + I)$.

Warning: orders > 1 may cause issues if Bset is passed, untested

Parameters

- **B** (`ndarray`) – Input signal array (`n_vertices`, `n_timesteps`).
- **scales** (`list[float]`, *default*: `[1]`) – List of scales to compute coefficients for.
- **Bset** (`csc_matrix`, *optional*) – Sparse indicator vector for localized coefficient computation.
- **refactor** (`bool`, *default*: `True`) – Whether to perform numeric factorization for each scale.

Returns

Filtered signals for each scale.

Return type
list[np.ndarray]

6.1.2 Dynamic Convolution

class DyConvolve(*L*, *poles*)

Bases: `object`

Dynamic graph convolution context with efficient topology updates.

Optimized for graphs with evolving topologies where poles/scales remain constant. Pre-factors all shifted systems ($L + qI$) at initialization, then uses CHOLMOD's updown routines for efficient rank-1 updates when edges are added or removed.

Parameters

- **L** (`csc_matrix`) – Sparse Graph Laplacian of shape (`n_vertices`, `n_vertices`).
- **poles** (list[float] | VKernel) – Predetermined set of poles (equivalent to 1/scale for analytical filters).

➔ See also

Convolve

For graphs with constant topology.

Examples

```
>>> from sgwt import DyConvolve, LAPLACIAN_TEXAS_DELAY
>>> import numpy as np
>>> L = LAPLACIAN_TEXAS_DELAY
>>> signal = np.random.randn(L.shape[0], 100)
>>> poles = [0.1, 1.0, 10.0] # Pre-factor these shifted systems
>>> with DyConvolve(L, poles) as dconv:
...     coeffs = dconv.lowpass(signal)
...     dconv.addbranch(i=5, j=10, w=0.5) # Topology change
...     coeffs_new = dconv.lowpass(signal)
```

addbranch(*i*, *j*, *w*)

Adds a branch to the graph topology and updates all factorizations.

Uses CHOLMOD's updown routines for efficient rank-1 updates.

Parameters

- **i** (`int`) – Index of Vertex A.
- **j** (`int`) – Index of Vertex B.
- **w** (`float`) – Edge weight.

bandpass(*B*, *order=1*)

Computes band-pass filtered wavelet coefficients.

Uses the analytical form: $(4qL / (L + qI)^2)^{\text{order}}$.

Parameters

- **B** (`ndarray`) – Input signal array (`n_vertices`, `n_timesteps`).

- **order** (`int`, *default: 1*) – The order of the filter (number of times the operator is applied).

Returns

Filtered signals for each pre-defined pole.

Return type

`list[np.ndarray]`

convolve(*B*)

Performs graph convolution using the pre-defined kernel.

Parameters

B (`ndarray`) – Input signal array (`n_vertices`, `n_timesteps`) with column-major ordering (F).

Returns

Convolved signal (`n_vertices`, `n_timesteps`, `nDim`).

Return type

`ndarray`

highpass(*B*)

Computes high-pass filtered coefficients.

Uses the analytical form: $L / (L + qI)$.

Parameters

B (`ndarray`) – Input signal array (`n_vertices`, `n_timesteps`).

Returns

Filtered signals for each pre-defined pole.

Return type

`list[np.ndarray]`

lowpass(*B*, *Bset=None*, *order=1*)

Computes low-pass filtered scaling coefficients.

Uses the analytical form: $qI / (L + qI)$.

Parameters

- **B** (`ndarray`) – Input signal array (`n_vertices`, `n_timesteps`).
- **Bset** (`csc_matrix`, *optional*) – Sparse indicator vector for localized coefficient computation.

Returns

Filtered signals for each pre-defined pole.

Return type

`list[np.ndarray]`

6.1.3 Chebyshev Convolution

class ChebyConvolve(*L*)

Bases: `object`

Initializes a Chebyshev convolution context.

This context manager is used to perform graph convolutions via Chebyshev polynomial approximation. It estimates the spectral bound of the Laplacian upon initialization.

Parameters

L (`csc_matrix`) – Sparse Graph Laplacian.

convolve(*B*, *C*)

Performs graph convolution using Chebyshev polynomial approximation.

This method implements Clenshaw’s algorithm for the stable evaluation of the Chebyshev series on the graph signal *B*.

Parameters

- **B** (`ndarray`) – Input signal array of shape (n_vertices,) or (n_vertices, n_signals).
- **C** (`ChebyKernel`) – A `ChebyKernel` object containing the Chebyshev coefficients and the spectral bound of the approximation.

Returns

The convolved signal. The shape of the output is (n_vertices, n_signals, n_dims) for a 2D input *B*, or (n_vertices, n_dims) for a 1D input *B*. *n_dims* is the number of filter dimensions in the kernel.

Return type

`ndarray`

6.2 Applications

High-level application modules that leverage the underlying convolution engines for specific analysis tasks, such as joint spatial-temporal modal identification.

6.2.1 Spectral Graph Modal Analysis

class `SGMA`(*L*, *scales*, *freqs*, *order*=10, *w0*=2 * np.pi)

Bases: `object`

Spectral Graph Modal Analysis (SGMA) engine.

Encapsulates the logic for performing joint spatial-temporal wavelet transforms on graph signals to identify oscillatory modes. The SGMA decomposes a time-vertex signal into the wavenumber-frequency domain, enabling scale-dependent mode identification [1].

The joint wavelet transform is computed as:

$$m_{n,\tau}(\Lambda \times S) \approx L_n X R_\tau$$

where L_n is the SGWT localized at bus n , X is the time-vertex signal, and R_τ is the temporal wavelet matrix centered at time τ [2].

Parameters

- **L** (`csc_matrix`) – The graph Laplacian matrix of shape (n_buses, n_buses). Must be symmetric positive semi-definite.
- **scales** (`array_like`) – Spatial scales for the SGWT. Logarithmically spaced values are recommended (e.g., `np.geomspace(1e-3, 1e1, 150)`).
- **freqs** (`array_like`) – Temporal frequencies (in Hz) to analyze.
- **order** (`int`, *optional*) – Order of the bandpass filter. Default is 10.

Ts

Temporal scales (seconds) derived from frequencies and *w0*.

Type

`ndarray`

wavlen

Approximate wavelengths ($\sqrt{s}$) for each spatial scale.

Type

ndarray

poles

Poles for DyConvolve, computed as $1/\text{scale}$.

Type

list

 See also**DyConvolve**

Dynamic convolution context with pre-factored poles.

gaussian_wavelet

Temporal wavelet generating kernel.

Examples

```
>>> import numpy as np
>>> from sgwt import SGMA
>>> # L is a sparse graph Laplacian matrix
>>> scales = np.geomspace(1e-2, 1e1, 50)
>>> freqs = np.linspace(0.1, 2.0, 60)
>>> sgma = SGMA(L, scales=scales, freqs=freqs)
>>> # V is a signal matrix (buses x time), t is a time vector
>>> peaks = sgma.analyze(V, t, bus=0, time=5.0, top_n=5)
>>> sgma.close() # Release resources when done
```

analyze(V, t, bus, time, top_n=5, min_dist=5)

Perform a full SGMA analysis for a single bus.

This is a convenience method that combines computing the spectrum and finding peaks.

Parameters

- **V** (ndarray) – Signal matrix of shape (n_buses, n_time).
- **t** (ndarray) – Time vector of shape (n_time,).
- **bus** (int) – Bus index for localized analysis.
- **time** (float) – The time instant (in seconds) to center the temporal wavelet.
- **top_n** (int, optional) – Maximum peaks to return. Default is 5.
- **min_dist** (int, optional) – Minimum index distance between peaks. Default is 5.

Returns

Peak information with keys: 'Wavelength', 'Frequency', 'Magnitude'.

Return type

dict

analyze_many(*V*, *t*, *time*, *buses=None*, *top_n=5*, *min_dist=5*, *verbose=True*)

Extract peaks from SGMA transforms across many specified buses.

Pre-computes $V @ B$ once and reuses it for all buses, significantly reducing computation time [1].

Parameters

- **V** (ndarray) – Signal matrix of shape (n_buses, n_time).
- **t** (ndarray) – Time vector of shape (n_time,).
- **time** (float) – The time instant (in seconds) to center the temporal wavelet.
- **buses** (list of int, optional) – List of bus indices to analyze. If None, all buses are used.
- **top_n** (int, optional) – Maximum peaks to return per bus. Default is 5.
- **min_dist** (int, optional) – Minimum index distance between peaks. Default is 5.
- **verbose** (bool, optional) – If True, prints progress updates. Default is True.

Returns

A named tuple containing: - **peaks**: dict with ‘Wavelength’, ‘Frequency’, ‘Magnitude’, ‘Bus_ID’. - **clusters**: dict with ‘Wavelength’, ‘Frequency’, ‘Density’.

Return type

NetworkAnalysisResult

close()

Release cached convolution resources.

Call this method when finished using the SGMA instance to free CHOLMOD memory allocations.

find_peaks(*spectrum*, *top_n=5*, *min_dist=5*)

Identify local maxima in the transform magnitude.

Uses morphological peak detection to find dominant modes in the wavelength-frequency plane.

Parameters

- **spectrum** (ndarray) – Spectrum magnitude of shape (n_scales, n_freqs).
- **top_n** (int, optional) – Maximum peaks to return. Default is 5.
- **min_dist** (int, optional) – Minimum index distance between peaks. Default is 5.

Returns

Peak information with keys:

- **Wavelength**: ndarray of spatial wavelengths (sqrt of scale)
- **Frequency**: ndarray of temporal frequencies in Hz
- **Magnitude**: ndarray of transform magnitudes at peaks

Return type

dict

Notes

Larger wavelengths correspond to inter-area oscillation modes, while smaller wavelengths indicate local oscillations [1].

spectrum(*V, t, bus, time, VB=None*)

Compute the SGMA spectrum magnitude at a specific bus and time.

Parameters

- **V** (ndarray) – Voltage signal matrix of shape (n_buses, n_time).
- **t** (ndarray) – Time vector of shape (n_time,).
- **bus** (int) – Bus index for localized analysis.
- **time** (float) – The time instant (in seconds) to center the temporal wavelet.
- **VB** (ndarray, optional) – Pre-computed $V @ B$ matrix. If provided, skips temporal matrix multiplication. Note: The provided VB must be computed for the given *time*.

Returns

Spectrum magnitude of shape (n_scales, n_freqs).

Return type

ndarray

6.3 Utility

Helper functions for signal generation, spectral estimation, and the definition of spectral kernels (rational and polynomial).

6.3.1 Utilities

This section covers the kernel objects used for defining filters and other helper functions.

Kernel Objects

class **VFKernel**(*R, Q, D*)

Vector Fitting Kernel representation.

A dataclass to store the components of a rational kernel approximation obtained from Vector Fitting.

classmethod **from_dict**(*data*)

Loads kernel data from a dictionary.

Parameters

data (dict) – A dictionary containing the kernel parameters, typically loaded from a JSON file. It should have ‘poles’ and ‘d’ keys.

Returns

A new instance of the VFKernel class.

Return type

VFKernel

D

Direct term (offset) of shape (n_dims,).

Q

Poles vector of shape (n_poles,).

R

Residue matrix of shape (n_poles, n_dims).

class `ChebyKernel`(*C*, *spectrum_bound*)

Stores Chebyshev polynomial approximations for one or more kernels.

classmethod `from_dict`(*data*)

Loads kernel data from a dictionary.

Parameters

data (`dict`) – Dictionary with a “spectrum_bound” key and an “approximations” key containing a list of {“coeffs”: [...]} objects.

Returns

A new instance of the `ChebyKernel` class.

Return type

`ChebyKernel`

classmethod `from_function`(*f*, *order*, *spectrum_bound*, *n_samples*=10000, *sampling*='quadratic', *min_lambda*=0.0)

Creates a `ChebyKernel` by fitting a vectorized function.

Parameters

- **f** (`Callable`[[`np.ndarray`], `np.ndarray`]) – The vectorized function to approximate.
- **order** (`int`) – Order of the Chebyshev polynomial to fit.
- **spectrum_bound** (`float`) – Upper bound of the function’s domain.
- **n_samples** (`int`, *default* 10000) – Number of points to sample.
- **sampling** (`str`, *default* 'quadratic') – Sampling strategy: ‘linear’ or ‘quadratic’. Quadratic sampling (t^2) clusters points near 0 to better capture sharp filter features.
- **min_lambda** (`float`, *default* 0.0) – The lower bound of the sampling range.

Returns

A new instance of the `ChebyKernel` class with the fitted coefficients.

Return type

`ChebyKernel`

classmethod `from_function_on_graph`(*L*, *f*, *order*, ***kwargs*)

Creates a `ChebyKernel` by fitting a function to a graph’s spectrum.

This is a convenience method that automatically estimates the spectral bound (*lambda_max*) of the graph Laplacian *L* before fitting.

Parameters

- **L** (`csc_matrix`) – The Graph Laplacian.
- **f** (`Callable`[[`np.ndarray`], `np.ndarray`]) – The vectorized function to approximate.
- **order** (`int`) – Order of the Chebyshev polynomial to fit.
- ****kwargs** – Additional arguments passed to `ChebyKernel.from_function`.

Returns

A new instance with the fitted coefficients.

Return type

`ChebyKernel`

evaluate(*x*)

Evaluates the Chebyshev approximation for this kernel.

Parameters

x (`ndarray`) – Points in the domain [0, spectrum_bound] to evaluate.

Returns

Evaluated function values at points in *x*.

Return type

`ndarray`

C

Coefficient matrix of shape (order + 1, n_dims).

spectrum_bound

Shared upper spectrum bound for all kernels.

Helper Functions

impulse(*lap*, *n=0*, *n_timesteps=1*)

Generates a Dirac impulse signal at a specified vertex.

Parameters

- **lap** (`csc_matrix`) – Graph Laplacian defining the number of vertices.
- **n** (`int`) – Index of the vertex where the impulse is applied.
- **n_timesteps** (`int`) – Number of time steps (columns) in the resulting signal.

Returns

(n_vertices, n_timesteps) array with 1.0 at index n and 0.0 elsewhere, in Fortran order.

Return type

`ndarray`

estimate_spectral_bound(*L*)

Estimates the largest eigenvalue (spectral bound) of a matrix.

This is typically used to find the domain [0, lambda_max] for Chebyshev polynomial approximations.

Parameters

L (`csc_matrix`) – The matrix (e.g., Graph Laplacian) for which to estimate the bound.

Returns

An estimate of the largest eigenvalue, scaled by 1.01 for safety.

Return type

`float`

6.3.2 Math Functions

This module provides scalar implementations of common analytical filter functions used in Spectral Graph Signal Processing. These are useful for generating target functions for polynomial or rational approximations.

Low-Pass

$$\phi_s(\lambda) = \frac{1}{s\lambda + 1}$$

lowpass(x , $scale=1.0$)

Computes the spectral response of a low-pass filter.

Analytical form: $1 / (s*x + 1)$

Parameters

- **x** (`ndarray`) – Input array of eigenvalues (or frequencies).
- **scale** (`float`, *default: 1.0*) – The scale of the filter.

Returns

The filter's gain at each point in x .

Return type

`ndarray`

High-Pass

$$\mu_s(\lambda) = \frac{s\lambda}{s\lambda + 1}$$

highpass(x , $scale=1.0$)

Computes the spectral response of a high-pass filter.

Analytical form: $s*x / (s*x + 1)$

Parameters

- **x** (`ndarray`) – Input array of eigenvalues (or frequencies).
- **scale** (`float`, *default: 1.0*) – The scale of the filter.

Returns

The filter's gain at each point in x .

Return type

`ndarray`

Band-Pass

$$\psi_s(\lambda) = \frac{4s\lambda}{(s\lambda + 1)^2}$$

bandpass(x , $scale=1.0$, $order=1$)

Computes the spectral response of a band-pass filter.

This is based on the SGWT wavelet design. Analytical form: $((4/s * x) / (x + 1/s)^2)^{order}$

Parameters

- **x** (`ndarray`) – Input array of eigenvalues (or frequencies).
- **scale** (`float`, *default: 1.0*) – The scale of the filter.
- **order** (`int`, *default: 1*) – The order of the filter.

Returns

The filter's gain at each point in x .

Return type

`ndarray`

Gaussian Wavelet

gaussian_wavelet(*time*, *a*=1, *b*=0, *w0*=1)

Compute a Morlet-like Gaussian wavelet.

Generates a complex-valued wavelet with Gaussian envelope and oscillatory carrier, normalized for continuous wavelet transform applications.

Parameters

- **time** (ndarray) – Time vector of shape (n_time,) in seconds. Must have at least 2 elements for timestep inference.
- **a** (float, default 1) – Scale parameter (temporal dilation). Larger values produce wider wavelets centered on lower frequencies.
- **b** (float, default 0) – Translation parameter (center time in seconds).
- **w0** (float, default 1) – Central angular frequency in rad/s. Default of 2π gives 1 Hz center frequency.

Returns

Complex-valued wavelet of shape (n_time,), normalized by dt/a .

Return type

ndarray

Examples

```
>>> import numpy as np
>>> from sgwt.functions import gaussian_wavelet
>>> t = np.linspace(0, 10, 1000)
>>> psi = gaussian_wavelet(t, a=0.5, b=5.0, w0=2*np.pi)
```

DEVELOPMENT

This section provides resources for developers, including instructions for running the validation test suite and the project's version history.

7.1 Validation Tests

The test suite verifies the correctness of `sgwt` installations. All tests achieve 100% code coverage.

7.1.1 Test Modules

- **test_cholconv.py**: Static and dynamic graph convolution (`Convolve`, `DyConvolve`), VF kernels, topology updates.
- **test_chebyshev.py**: Chebyshev polynomial approximation and `ChebyConvolve`.
- **test_functions.py**: Analytical filter functions (low-pass, band-pass, high-pass).
- **test_util.py**: Resource loading, DLL utilities, `ChebyKernel`, `VFKernel`.

7.1.2 Running Tests

Install test dependencies:

```
pip install sgwt[test]
```

Run from source checkout:

```
pytest
```

Run on installed package:

```
pytest --pyargs sgwt.tests
```

7.2 Changelog

All notable changes to this project will be documented in this file.

The format is based on [Keep a Changelog](#), and this project adheres to [Semantic Versioning](#).

7.2.1 [0.3.5] - 2026-01-10

Changed

- Expanded test suite to full code and branch coverage, including all edge cases and defensive branches.
- Tests to do cover KLU wrapper, because it is not used in this version.

7.2.2 [0.3.4] - 2026-01-06

Changed

- Migrated the entire test suite from `unittest` to `pytest` for a more modern, readable, and feature-rich testing framework. The tests are now included as an installable sub-package `sgwt.tests`.

Fixed

- Resolved an issue where `DyConvolve.addbranch` would cause an unhandled error for negative branch weights. It now raises a descriptive `ValueError`.

7.2.3 [0.3.3] - 2026-01-05

Added

- New documentation section for Chebyshev approximation examples and benchmarks.
- Lazy loading for built-in resources (Laplacians, Signals, Kernels) to improve import time and reduce memory usage.

Fixed

- Offset `d_a` of Kernel Fitted approximation correctly implemented.
- Documentation typos referring to `VFKern` instead of `VFKernel`.

7.2.4 [0.3.2] - 2026-01-03

Added

- `CHANGELOG.rst` to track project history.
- `CITATION.cff` for easier citation.

Changed

- Renamed internal `io` module to `util` for better semantic clarity.
- Renamed `VFKern` to `VFKernel` for clarity and consistency.
- Improved `README.rst` layout, usage example, and author links.
- Enhanced documentation for the data library (`library_data.rst`) with code snippets.
- Corrected and clarified kernel JSON format documentation (`library_json.rst`).
- Standardized API and documentation for consistency (e.g., `n_vertices`, `np.ndarray` links, default parameter formatting).
- Switched documentation parser from `numpydoc` to `sphinx.ext.napoleon` for improved styling and stability.

Fixed

- Updated documentation and packaging to reflect Windows-only compatibility due to the pre-compiled `CHOLMOD.dll`.

7.2.5 [0.3.1] - 2026-01-01

Added

- First stable public release of the sgwt package.
- Comprehensive documentation, usage examples, and testing suite.

GLOSSARY

Adjacency Matrix

Matrix $\mathbf{W} \in \mathbb{R}^{N \times N}$ where W_{ij} is the edge weight between vertices i and j . Zero entries indicate no edge.

Degree Matrix

Diagonal matrix $\mathbf{D} \in \mathbb{R}^{N \times N}$ where $D_{ii} = \sum_j W_{ij}$ is the sum of weights connected to vertex i .

Graph Laplacian

The matrix $\mathbf{L} = \mathbf{D} - \mathbf{W}$ encoding graph structure. Its eigenvalues provide a notion of graph frequency, with physical interpretation depending on the branch weighting scheme.

Poles

Shift values q in $(\mathbf{L} + q\mathbf{I})^{-1}$. For analytical filters, pole = 1/scale. In `VFKernel`, stored as `Q` with shape `(n_poles,)`.

Rational Approximation

Representing a filter function as a ratio of polynomials. Vector Fitting produces pole-residue form $g(\lambda) \approx d + \sum_k \frac{r_k}{\lambda + q_k}$, enabling efficient linear solves.

Residues

Complex coefficients r_k in a rational approximation. In `VFKernel`, stored as `R` with shape `(n_poles, n_dims)` where `n_dims` is the output dimension of the kernel.

Scale

Dilation parameter s controlling filter bandwidth. Related to poles by $s = 1/q$ and to wavelength by $\lambda \approx \sqrt{s}$. Larger scales correspond to lower graph frequencies.

Spectral Domain

The eigenspace of the Graph Laplacian. Signals are decomposed into eigenvector components, analogous to Fourier analysis on regular domains.

Vector Fitting

Algorithm for fitting a rational function to frequency-domain samples. Produces poles and residues stored in `VFKernel` for use with `Convolve` and `DyConvolve`.

Vertex Domain

Signals defined on graph nodes as vectors $\mathbf{f} \in \mathbb{R}^N$, where the i -th element is the signal value at vertex i . Contrast with *Spectral Domain*.

VFKernel

Data structure holding Vector Fitting results:

- `Q`: poles, shape `(n_poles,)`
- `R`: residues, shape `(n_poles, n_dims)`
- `D`: direct term, shape `(n_dims,)`

Wavelength

Spatial extent of oscillation modes, approximately $\sqrt{s}$ where s is the scale. Large wavelengths indicate spatially extended (inter-area) modes; small wavelengths indicate localized modes.

Wavenumber

Spatial frequency k . For appropriately weighted graphs (e.g., inverse squared distance), Laplacian eigenvalues correspond to k^2 , linking spectral and physical domains.

A

addbranch() (*DyConvolve method*), 37
 Adjacency Matrix, **51**
 analyze() (*SGMA method*), 40
 analyze_many() (*SGMA method*), 40

B

bandpass() (*Convolve method*), 35
 bandpass() (*DyConvolve method*), 37
 bandpass() (*in module sgwt.functions*), 45

C

C (*ChebyKernel attribute*), 44
 ChebyConvolve (*class in sgwt*), 38
 ChebyKernel (*class in sgwt*), 42
 close() (*SGMA method*), 41
 Convolve (*class in sgwt*), 35
 convolve() (*ChebyConvolve method*), 39
 convolve() (*Convolve method*), 36
 convolve() (*DyConvolve method*), 38

D

D (*VFKernel attribute*), 42
 Degree Matrix, **51**
 DyConvolve (*class in sgwt*), 37

E

estimate_spectral_bound() (*in module sgwt*), 44
 evaluate() (*ChebyKernel method*), 43

F

find_peaks() (*SGMA method*), 41
 from_dict() (*ChebyKernel class method*), 43
 from_dict() (*VFKernel class method*), 42
 from_function() (*ChebyKernel class method*), 43
 from_function_on_graph() (*ChebyKernel class method*), 43

G

gaussian_wavelet() (*in module sgwt.functions*), 46
 Graph Laplacian, **51**

H

highpass() (*Convolve method*), 36
 highpass() (*DyConvolve method*), 38
 highpass() (*in module sgwt.functions*), 45

I

impulse() (*in module sgwt*), 44

L

lowpass() (*Convolve method*), 36
 lowpass() (*DyConvolve method*), 38
 lowpass() (*in module sgwt.functions*), 44

P

Poles, **51**
 poles (*SGMA attribute*), 40

Q

Q (*VFKernel attribute*), 42

R

R (*VFKernel attribute*), 42
 Rational Approximation, **51**
 Residues, **51**

S

Scale, **51**
 SGMA (*class in sgwt*), 39
 Spectral Domain, **51**
 spectrum() (*SGMA method*), 41
 spectrum_bound (*ChebyKernel attribute*), 44

T

Ts (*SGMA attribute*), 39

V

Vector Fitting, **51**
 Vertex Domain, **51**
 VFKernel, **51**
 VFKernel (*class in sgwt*), 42

W

Wavelength, [52](#)

Wavenumber, [52](#)

wavlen (*SGMA attribute*), [39](#)